

FACIAL EXPRESSION RECOGNITION USING CONVOLUTIONAL NEURAL NETWORK

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

Submitted By

Kancharapoo Rajesh

(18981A0570)

Mathsa Vign Dinesh

(18981A0591)

Mohammed Abdul Rahman

(18981A0596)

UNDER THE ESTEEMED GUIDENCE OF

B.H.V.S. Narayana

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



RAGHU ENGINEERING COLLEGE

(AUTONOMOUS)

Affiliated to JNTU KAKINADA

Approved by AICTE, Accredited by NBA, Accredited by NAAC with A grade

www.raghuengineering.com

2021-2022

RAGHU ENGINEERING COLLEGE

(AUTONOMOUS)

Affiliated to JNTU KAKINADA

Approved by AICTE, Accredited by NBA, Accredited by NAAC with A grade

www.raghuengineering.com

2020-2021



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that this project entitled “**Facial Expression Recognition Using convolutional neural network**” done by “ **Kancharapoo Rajesh (18981A0570), Mathsa Vign Dinesh (18981A0591), Mohammed Abdul Rahman (18981A0596)** ” are students of B.Tech in the Department of Computer Science and Engineering, Raghu Engineering College, during the period 2018-2022, in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering to the Jawaharlal Nehru Technological University, Kakinada is a record of bonafide work carried out under my guidance and supervision.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree.

Internal Guide

B.H.V.S. Narayana
Dept of CSE,
Raghu Engineering College,
Dakamarri (V),
Visakhapatnam.

Head of the Department

Sri S. SRINADH RAJU,
Dept of CSE,
Raghu Engineering College
Dakamarri (V),
Visakhapatnam.

EXTERNAL EXAMINER

DISSERTATION APPROVAL SHEET

This is to certify that the dissertation titled

FACIAL EXPRESSION RECOGNITION USING CONVOLUTIONAL NEURAL NETWORK

BY

Kancharapoo Rajesh

(18981A0570)

Mathsa Vign Dinesh

(18981A0591)

Mohammed Abdul Rahman

(18981A0596)

*Is approved for the degree of **Bachelor of Technology***

B.H.V.S. Narayana

Assistant Professor

Internal Examiner

External Examiner

HOD

Date: 24/03/2022

DECLARATION

This is to certify that this project titled **“FACIAL EXPRESSION RECOGNITION USING CONVOLUTIONAL NEURAL NETWORK”** is bonafied work done by my team, In partial fulfillment of the requirements for the award of the degree B.Tech and submitted to the **Department of Computer Science and Engineering, Raghu Engineering College, Dakamarri.**

We also declare that this project is a result of our own effort and that has not been copied from anyone and we have taken only citations from the sources which are mentioned in the references.

This work was not submitted earlier at any other University or Institute for the reward of any degree.

Date: 24/03/2022

Place: Visakhapatnam, India

Kancharapoo Rajesh

(18981A0570)

Mohammed Abdul Rahman

(18981A0596)

Mathsa Vign Dinesh

(18981A0591)

ACKNOWLEDGEMENT

We express our sincere gratitude to my esteemed Institute “Raghu Engineering College”, which has provided us an opportunity to fulfill the most cherished desire to reach my goal.

We take this opportunity with great pleasure to put on record our ineffable personal indebtedness to **Mr.Raghu Kalidindi, Chairman of Raghu Engineering College** for providing necessary departmental facilities.

We would like to thank the Principal **Dr. CH. Srinivasu**, Administration and Management of “**Raghu Engineering College**”, for providing the requisite facilities to carry out project in campus.

Our sincere thanks to Professor Sri **S.Srinadh Raju, Head of the Department**, Department of Computer Science and Engineering, Raghu Engineering College, for his kind support in the successful completion of this work.

We sincerely express our deep sense of gratitude to **B.H.V.S. Narayana, Assistant Professor**, Department of Computer Science and Engineering, Raghu Engineering College, for his perspicacity, wisdom and sagacity coupled with compassion and patience. It is our great pleasure to submit this work under his wing.

We extend our deep hearted thanks to all faculty members of Computer Science department for their value based imparting of theory and practical subjects, which were used in the project.

We are thankful to the non-teaching staff of the Department of Computer Science and Engineering, Raghu Engineering College, for their inexpressible support.

Regards

Kancharapoo Rajesh (18981A0570)

Mathsa Vign Dinesh (18981A0591)

Mohammed Abdul Rahman(18981A0596)

ABSTRACT

The emotions, set in simple words are what people feel. Emotional aspects have huge impact on Social intelligence like communication understanding, decision making and helps in understanding behavioral aspect of human.

Human faces provide various information about emotions. As per psychological researcher, a person expresses his emotions less by verbal talk and more by non-verbal body posture and gestures. Emotion recognition or Affective Computing (AC) being the AI related area imparts intelligence to computers in recognizing human emotions. Emotion recognition is proved a popular research area topic in few decades.

Facial expression for emotion detection has always been an easy task for humans, but achieving the same task with a computer algorithm is quite challenging. With the recent advancement in computer vision and machine learning, it is possible to detect emotions from images.

Keywords: Tensorflow, Keras, CNN, Facial expression Recognition (FER)

TABLE OF CONTENTS

CONTENT	PAGE NUMBER
Certificate	i
Dissertation Approval Sheet	ii
Declaration	iii
Acknowledgement	iv
Abstract	v
Contents	vi
List of figures	viii
CHAPTER 1: INTRODUCTION	10
1.1 Purpose	11
1.2 Scope	12
1.3 Motivation	
1.4 Fundamental concepts	13
1.4.1 Techniques	15
1.4.1.1 Deep Learning	
1.4.2 Techniques for Recognizing facial Expressions	
1.4.2.1 Image Preprocessing	
1.4.2.2 CNN	
1.5 Proposed Algorithms	16
1.6 Proposed System	17
1.7 Overview	18
CHAPTER 2: LITERATURE SURVEY	19
2.1 Introduction to Literature Survey	20
2.2 Literature Survey	21
CHAPTER 3: SYSTEM ANALYSIS	24
3.1 Introduction	25
3.2 Problem statement	
3.3 Existing System	
3.4 Modules Description	26

3.4.1 Keras	
3.4.2 Sklearn	
3.4.3 Seaborn	
3.4.4 Mathplotlib	
3.4.5 PIL	
3.4.6 Numpy	
3.4.7 Pandas	
3.4.8 Glob	
3.4.9 Itertools	
CHAPTER 4: SYSTEM REQUIREMENTS SPECIFICATION	28
4.1 Software Requirements	29
4.2 Hardware Requirements	
CHAPTER 5: SYSTEM DESIGN	30
5.1 Introduction	31
5.2 System Model	
5.2.1 Convolutional-Layer Representation	33
5.2.2 Spatial Arrangement	34
5.2.3 Convolutional Operational Implementation	36
5.2.4 ReLU Functionality	
5.2.5 Pooling Layer	37
5.3 System Architecture	39
5.4 UML Diagrams	41
5.4.1 Use Case Diagrams	42
5.4.2 Class Diagram	43
5.4.3 Sequence Diagram	44
5.4.4 State Chart Diagram	45
CHAPTER 6: IMPLEMENTATION	46
6.1Technology Description	47
6.1.1 OpenCV	
6.1.2 numpy	48
6.1.3 Keras	49

6.1.4 Tensorflow	
6.2 Sample code	50
CHAPTER 7: OUTPUT SCREENS	53
7.1 Output Screenshots	54
7.2 Model Training Output	55
CHAPTER 8: TESTING	56
8.1 Introduction to Testing	57
8.2 Types of Testing	58
8.2.1 Unit testing	
8.2.2 Integration Testing	
8.2.3 Regression Testing	
8.2.4 System Testing	
8.2.5 Performance Testing	
8.2.6 Alpha Testing	
8.2.7 Beta Testing	
8.3 Sample Test cases	60
8.3.1 Sample Code	
8.3.2 Sample Output	61
CHAPTER 9: CONCLUSION AND FUTURE ENHANCEMENTS	64
9.1 Conclusion and Further Enhancement	65
REFERENCES	66
PAPER PUBLICATION	70

LIST OF FIGURES

CONTENT	PAGE NUMBER
1. Figure – 5.1 CONVOLUTIONAL-LAYER REPRESENTATION	34
2. Figure – 5.2 SPATIAL ARRANGEMENT	35
3. Figure – 5.3 CONVOLUTION OPERATION IMPLEMENTATION	37
4. Figure – 5.4 ReLU FUNCTIONALITY	37
5. Figure – 5.5 POOLING LAYER	38
6. Figure – 5.6 CNN ARCHITECTURE	40
7. Figure – 5.7 CNN MODEL EVALUATION	41
8. Figure – 5.8 USE CASE DIAGRAM	43
9. Figure – 5.9 CLASS DIAGRAM	44
10. Figure – 5.10 SEQUENCE DIAGRAM	45
11. Figure – 5.11 ACTIVITY DAIGRAM	46
12. Figure – 8.1 SAMPLE TEST CASES TABLE	64

CHAPTER-1

INTRODUCTION

Humans interact with each other mainly through speech, but also through body gestures, to emphasize certain parts of their speech and to display emotions. One of the important ways humans display emotions is through facial expressions which are a very important part of communication. Though nothing is said verbally, there is much to be understood about the messages we send and receive through the use of nonverbal communication. Facial expressions convey non verbal cues, and they play an important role in interpersonal relations. Automatic recognition of facial expressions can be an important component of natural human-machine interfaces; it may also be used in behavioral science and in clinical practice. Although humans recognize facial expressions virtually without effort or delay, reliable expression recognition by machine is still a challenge. There have been several advances in the past few years in terms of face detection, feature extraction mechanisms and the techniques used for expression classification, but development of an automated system that accomplishes this task is difficult. we present an approach based on Convolutional Neural Networks (CNN) for facial expression recognition. The input into our system is an image; then, we use CNN to predict the facial expression label which should be one these labels: anger, happiness, fear, sadness, disgust and neutral.

1.1 PURPOSE

Human-computer interaction technology refers to a kind of technology which takes computer equipment as the medium, so as to realize the interaction between human and computer. In the recent years, with the rapid development of pattern recognition and artificial intelligence, more and more research has been conducted in the field of human-computer interaction technology [1,2]. The facial expression recognition, as an important means of intelligent human-computer interaction, has a broad application background. It has been applied in the fields of assistant medicine, distance education, interactive games and public security [3-5]. The facial expression recognition extracts the information representing the facial expression features from the original input facial expression images through computer image processing technology, and classifies the facial expression features according to human emotional expression, such as happiness, surprise, aversion and neutrality [6,7]. The facial expression recognition plays an important role in the research of emotional quantification. Under the trend of artificial intelligence, the communication between human and computer becomes

easier and easier. Therefore, vigorously promoting the research of facial expression recognition technology is of great value to the development of individuals and society [8,9].

1.2 SCOPE

MEDICAL REASEARCH INTO AUTISM

Machine learning systems for facial emotion recognition are particularly suited for the study of autism spectrum disorder (ASD), where sufferers have developmental and long-term difficulties in evaluating facial emotions.

MARKET RESEARCH

Since monitoring user data by stealth has become such a political football in recent years, facial expression recognition technologies are on their strongest ground where the subjects are willing accomplices, such as in focus groups and other forms of beta-testing for product marketing.

RECRUITMENT

The rise of teleconferencing-based interaction since the advent of COVID-19 solves two major obstacles to using FER in a recruitment interview context: the negative effects¹⁹ of explicitly training a camera on someone you are talking to (solved by the user's webcam); and the more recent challenge of the subject's face being occluded by a mask and thus hindering intelligent video analysis.

1.3 MOTIVATION

For developing this project, my motivation was the ideology of the implementation of CNN techniques and usage of neural networks. The facial expression recognition, as an important means of intelligent human-computer interaction, has a broad application background. It has been applied in the fields of assistant medicine, distance education, interactive games and public security [3-5]. The idea of image preprocessing is fascinating and an interesting concept to be learned.

1.4 FUNDAMENTAL CONCEPTS

The fundamental concepts used in this project is the ideology and techniques of deep learning. Deep learning is based on the branch of machine learning, which is a subset of artificial intelligence. Since neural networks imitate the human brain and so deep learning will do. In deep learning, nothing is programmed explicitly. Basically, it is a machine learning class that makes use of numerous nonlinear processing units so as to perform feature extraction as well as transformation. The output from each preceding layer is taken as input by each one of the successive layers.

So basically, deep learning is implemented by the help of deep networks, which are nothing but neural networks with multiple hidden layers.

Architectures

- **Deep Neural Networks:** It is a neural network that incorporates the complexity of a certain level, which means several numbers of hidden layers are encompassed in between the input and output layers. They are highly proficient on model and process non-linear associations.
- **Deep Belief Networks:** A deep belief network is a class of Deep Neural Network that comprises of multi-layer belief networks.

Steps to perform DBN:

- With the help of the Contrastive Divergence algorithm, a layer of features is learned from perceptible units.
 - Next, the formerly trained features are treated as visible units, which perform learning of features.
 - Lastly, when the learning of the final hidden layer is accomplished, then the whole DBN is trained.
- **Recurrent Neural Networks:** It permits parallel as well as sequential computation, and it is exactly similar to that of the human brain (large feedback network of connected neurons). Since they are capable enough to reminisce all of the imperative things related to the input they have received, so they are more precise.

Types of Deep Learning Networks

- **Feed Forward Neural Network**

A feed-forward neural network is none other than an Artificial Neural Network, which ensures that the nodes do not form a cycle. In this kind of neural network, all the perceptrons are organized within layers, such that the input layer takes the input, and the output layer generates the output.

- **Recurrent Neural Network**

Recurrent neural networks are yet another variation of feed-forward networks. Here each of the neurons present in the hidden layers receives an input with a specific delay in time. The Recurrent neural network mainly accesses the preceding info of existing iterations. For example, to guess the succeeding word in any sentence, one must have knowledge about the words that were previously used.

- **Convolutional Neural Network**

Convolutional Neural Networks are a special kind of neural network mainly used for image classification, clustering of images and object recognition. DNNs enable unsupervised construction of hierarchical image representations. To achieve the best accuracy, deep convolutional neural networks are preferred more than any other neural network.

- **Restricted Boltzmann Machine**

RBMs are yet another variant of Boltzmann Machines. Here the neurons present in the input layer and the hidden layer encompasses symmetric connections amid them. However, there is no internal association within the respective layer. But in contrast to RBM, Boltzmann machines do encompass internal connections inside the hidden layer. These restrictions in BMs helps the model to train efficiently.

- **Autoencoders**

An autoencoder neural network is another kind of unsupervised machine learning algorithm. Here the number of hidden cells is merely small than that of the input cells. But the number of input cells is equivalent to the number of output cells. An autoencoder network is trained to display the output similar to the fed input to force AEs to find common patterns and generalize the data. The autoencoders are mainly used for the smaller representation of the input. It helps in the reconstruction of the original data from compressed data. This algorithm is comparatively simple as it only necessitates the output identical to the input.

1.4.1 TECHNIQUES

1.4.1.1 DEEP LEARNING

Deep learning (also known as deep structured learning or hierarchical learning) is part of a broader family of machine learning methods based on artificial neural networks. Learning can be supervised, semi-supervised or unsupervised. Deep learning architectures such as deep neural networks, deep belief networks, recurrent neural networks and Convolution Neural Networks have been applied to fields including computer vision, speech recognition, natural language processing, audio recognition, social network filtering, machine translation, bioinformatics, drug design, medical image analysis, material inspection and board game programs, where they have produced results comparable to and in some cases superior to human expert.

1.4.2 TECHNIQUES FOR RECOGNIZING FACIAL EXPRESSIONS

1.4.1 IMAGE PREPROCESSING

Pre-processing is a common name for operations with images at the lowest level of abstraction - both input and output are intensity images. These iconic images are of the same kind as the original data captured by the sensor, with an intensity image usually represented by a matrix of image function values (brightness). The aim of pre-processing is an improvement of the image data that suppresses unwilling distortions or enhances some image features important for further

processing, although geometric transformations of images (e.g. rotation, scaling, translation) are classified among pre-processing methods here since similar techniques are used. Image pre-processing methods are classified into four categories according to the size of the pixel neighborhood that is used for the calculation of a new pixel brightness. This deals with pixel brightness transformations, Section 4.2 describes geometric transformations. It considers preprocessing methods that use a local neighborhood of the processed pixel and briefly characterizes image restoration that requires knowledge about the entire image.

1.4.2 CONVOLUTIONAL NEURAL NETWORK

Convolutional Neural Networks are a special type of feed-forward artificial neural network in which the connectivity pattern between its neuron is inspired by the visual cortex. The visual cortex encompasses a small region of cells that are region sensitive to visual fields. In case some certain orientation edges are present then only some individual neuronal cells get fired inside the brain such as some neurons responds as and when they get exposed to the vertical edges, however some responds when they are shown to horizontal or diagonal edges, which is nothing but the motivation behind Convolutional Neural Networks. The Convolutional Neural Networks, which are also called as covnets, are nothing but neural networks, sharing their parameters.

1.5 PROPOSED ALGORITHM

For our project, we will be implementing the CNN algorithm. A convolutional neural network, or CNN, is a deep learning neural network sketched for processing structured arrays of data such as portrayals. CNN are very satisfactory at picking up on design in the input image, such as lines, gradients, circles, or even eyes and faces. This characteristic that makes convolutional neural network so robust for computer vision. CNN can run directly on a underdone image and do not need any preprocessing. A convolutional neural network is a feed forward neural network, seldom with up to 20. The strength of a convolutional neural network comes from a particular kind of layer called the convolutional layer. With three or four convolutional layers it is viable to recognize handwritten digits and with 25 layers it is possible to differentiate human faces. The agenda for this sphere is to activate machines to view the world as humans do, perceive it in alike

fashion and even use the knowledge for a multitude of duty such as image and video recognition, image inspection and classification, media recreation, recommendation systems, natural language processing, etc.

1.6 PROPOSED SYSTEM

The goal of our Project is to build an accurate model using “Convolution Neural Networks” Algorithm. The goal is to identify one of the seven different facial expressions from the face image. Subsequently training of the model is done and tested.

ADVANTAGES OF PROPOSED SYSTEM:

- Over fitting of model is reduced.
- Accuracy is improved and accuracy of our model is 85%.
- Model building takes less time.

1.7 OVERVIEW

Facial Expressions are an essential feature of non- verbal communication, as we move towards the digitization the human computer interactions plays a vital role. The emotional changes results change in the expressions. This paper elaborates development of Deep Convolutional Neural Network Model using tf.keras for building and training Deep Learning Model. The aim is to classify facial image into one of the seven face detection classifiers using open CV and one of its classifiers for drawing the boundary box around the face to detect the correct expression.

CHAPTER-2

LITERATURE SURVEY

2.1 INTRODUCTION TO LITERATURE SURVEY

A literature survey is a piece of discursive prose, not a list describing or summarizing one piece of literature after another. It is an iterative process, assessing and distilling information. One of the key purposes of the literature survey is to investigate a problem that no one else has addressed. We use a wide range of sources to review that explain the rationale, places the research in a broad context, reviews studies that can be generalized and are transferable, and identifies methodological shortcomings. We follow an iterative process of conducting a preliminary search on the right databases before screening the results and developing the conceptual framework. The right approach yields an actionable plan for empirical investigation and analysis. The final report includes a coherent synthesis in the form of a descriptive report containing text, graphs, and tables. Spreadsheets or grids to help track numerous sources across consistent variables or metrics are also provided. Checklists to help track connections and progress are updated daily to keep our customers abreast of the progress in their research. The focus of a literature survey is on the following and in this order:

- Existing theories on a topic with universal acceptance across the board books on the subject acting as a reference for the concepts that project uses whether they are specific or generic.
- Current research concerning the field of the project from the oldest to latest. Research papers might be a reference for theories but most cases require a critical comparison to establish the purpose of the project and improvement.
- You may also include another project report and what helped you with challenges for the project and by ongoing work if it is available

Literature surveys provide brief overviews or a summary of the current research on topics. The structure written requires to be in a way that it seemed logical. It needs to chronologically represent a development of the ideas in the field that is being researched. The length of a literature survey depends much on whether the purpose of the project report is to complete a college assignment or submitting for journal publication. It can review a few research papers on a topic or be a full-length discussion on the significant work in the field until that date.

2.2 LITERATURE SURVEY

The purpose of performing this systematic literature review was to select and categorize the best available approaches facial expression recognition using neural networks (NNs). Systematic literature reviews collect and analyze existing studies according to predefined evaluation criteria. Such reviews help to determine what is already known in the concerned domain of study [10]. All data collected from primary sources are organized and analyzed. Once systematic literature is completed, it provides a more sensible, logical, and robust answer to the underlying question of the research [11]. The population of studies considered in the current systematic literature review consisted of research papers relevant to SC detection based on deep neural network (DNN) techniques.

In recent years various architectures and models of CNN are created and used for facial expression recognition. This research includes a summery review of researches related to facial expression recognition using CNN.

A. Deep CNN for FER

AyşegülUçar [5] proposed a CNN model with 10 layers for FER as shown in Fig.3. In their architecture, a convolutional layer with kernel size $5 * 5$, stride 1 and pad 2 was applied. Next a max pooling layer with kernel size $3 * 3$, stride 2 and pad 1 is applied. This process is repeated 3 times with different strides and pads as shown in Fig. 3. Followed by a convolutional layer with kernel size $2 * 2$, stride 1 and pad 0 is applied which is followed by another convolution layer of kernel size $1 * 1$, stride 1 and pad 0. Finally a fully connected layer is added to the network. Their model was evaluated on images from JAFFE database [6] and CK database [7]. In first evaluation seven facial expressions of various images from the JAFFE database were used for training. In the second evaluation Cohn-Kanadedatabase which contains images of all races was employed for six expressions. They resized the images to 16x16 and the proposed model was trained for 30 epochs with a batch size of 40. They used 0.001 as the learning rate for first eleven epochs, 0.002 for 13-29 epochs and 0.00001 for the last epoch. Their results shows that their proposed method outperforms traditional geometric and appearance based methods with high accuracy.

B. Baseline CNN structure for FER

Minchul Shin et al. [8] have looked into four network models that are known to perform well in facial

expression recognition to analyse the most efficient network structure. The effect of techniques for preprocessing the input image on the performance was also investigated. The first network that was looked into was Tang's CNN structure [9]. It contains a layer for input transformation and three convolutional and pooling layers and a fully connected two-layer perceptron at the end. The second network is Yu's structure [2] which contains five convolutional layers, three stochastic pooling layers, and three fully connected layers. The network has two convolutional layers prior to pooling, except for the first layer. The third network that is investigated is Kahou's structure [3] which contains three convolutional and pooling layers followed by a MLP of two layers. The last one is the Caffe-Image Net structure [10]. It was designed for the classification of images taken from the ImageNet dataset into 1000 classes. But the output nodes were reduced to seven in Baseline CNN approach. Every convolutional and fully connected layers of all the four networks is applied with a ReLU layer and a dropout layer. Five test sets (FET – 2013, SFEW2.0, CK+, KDEF and Jaffe) are chosen to perform tests with the four network structures. For the pre-processing of input image it was found that the Histogram equalization method shows the most reliable performance for all the four networks. It was also observed that Tang's network could achieve reasonably high accuracy for histogram equalized images compared to the other network models. Based on this observation they suggested Tang's simple network along with histogram equalization as the baseline model for carrying out further research.

C. FER with CNN ensemble

Kaung Liu et al. [11] have proposed a model consisting of many subnets that are structured differently. Each of these subnets are separately trained on a training set and combined together. The output layers are removed and the layers before the last layer are concatenated together. Finally, this connected network is trained to output the final expression labels. They evaluated their model using Facial Expression Recognition 2013 (FER – 2013) dataset. It contains grayscale image of faces of size 48 x 48 pixels. They divided the dataset into an 80 % training set and 20 % validation set. They trained the subnets separately. Each of the subnets achieved a different accuracy on the dataset. By combining and averaging the outputs of CNNs of different structures their network reports betterment in performance when compared to single CNN structure.

D. Stacked Deep Convolutional Auto-Encoders for Emotion Recognition from Facial Expressions

Ariel Ruiz-Garcia et al. [12] have studied the effect of reducing the number of convolutional layers and pretraining the Deep CNN as a Stacked Convolutional Auto-Encoder (SCAE) in a greedy layer-wise unsupervised fashion for emotion recognition using facial In the SCAE emotion recognition model, each convolutional layer and its subsequent layers: BN, ReLU and Max Pooling are treated as a single block and an Auto-Encoder is created for each one of these blocks. The first Auto-Encoder learns to reconstruct raw pixel data. The second Auto-Encoder learns to reconstruct the output of the first encoder and so on. Finally, the fully connected layer is trained to associate the output of the last convolutional encoder with its corresponding label. Their CNN with BN and the SCAE emotion recognizers are trained and tested using KDEF [13] dataset. Applying the pre-training technique to give weights to the CNN using Auto-Encoders increased their model's performance to 92.53% and dramatically reduced the training time. OBSERVATIONS From the above study it is clear that even though there are numerous approaches for facial expression recognition still models are being developed continuously. The reason for this is the accuracy. Researches are continuously trying to improve the accuracy of the FER by proposing various architectures and model. They also adopted some of the other techniques in their architectures to improve the accuracy as discussed in Section 1V to account for the problem of accuracy. Efforts are done to reduce the training time for better performance. Ensemble CNNs are used to improve the accuracy of facial expression recognition.

CHAPTER-3

SYSTEM ANALYSIS

3.1 INTRODUCTION

Facial Expression Recognition (FER) can be seen as a second step to face detection mechanism. Humans can express emotions through facial expressions which is a part of nonverbal communication. When a machine communicates with people, FER can give more affinities and personalized service to people depending on their emotions, which eventually increase the confidence and trust in people. Various ways in which we can express the emotions, such as facial expressions, voices, physiological signals, and text. Machines capture the expressions through camera, videos. Facial Expressions can be classified as: surprise, happy, neutral, angry, sad, disgust, and fear.

3.2 PROBLEM STATEMENT

The project deals with the recognition of facial expressions using CNN which is a deep learning technique and implementation of CNN algorithms. In our project we will be building a model which will classify the given input as one of the seven different types of expressions. Our project will take an image as an input and will tell which type of expression it is likely to be from the image. In this project, we will take an image of the human face and classify into one of the seven different expressions.

3.3 EXISTING SYSTEM

Haar functions is used for face,eyes and mouth detection,edge detection method for extracting the eyes correctly and bezier curves is applied to approximate the extracted regions. Then a set of distances for varied face type is extended and it is serve as training input for a multilayer neural network. The novel factor of this approach consists in applying bezier curves to efficiently extract the distances between facial parts. The preclassification is done using K-means algorithm. A two layered feed forward neural network created is then used as a classifying tool for the input images.The consistency of the results is demonstrated by the median value.The method is not able to treat situations when the eyes are closed.Strong illumination variations affect the results.

DRAWBACKS OF EXISTING ALGORITHMS:

- Over fitting of model occurs.
- Accuracy of the model is less.

3.4 MODULE DESCRIPTION

3.4.1 Keras

Keras is a minimalist Python library for deep learning that can run on top of Theano or TensorFlow. It was developed to make implementing deep learning models as fast and easy as possible for research and development.

3.4.2 Sklearn

Scikit-learn is a free machine learning library for Python. It features various algorithms like support vector machine, random forests, and k-neighbour, and it also supports Python numerical and scientific libraries like NumPy and SciPy. The sklearn library contains lot of efficient machine learning tools including classification, regression, clustering, and dimensionality reduction. It should not be used for reading the data manipulating the data and summarizing it. There are better libraries for that.(Numpy , Pandas, etc.)

3.4.3 Seaborn

Seaborn is a library for making statistical graphics in Python. It is built on top of matplotlib and closely integrated with pandas data structures. Seaborn aims to make visualization a central part of exploring and understanding data. Its dataset-oriented plotting functions operate on data frames and arrays containing whole datasets and internally perform the necessary semantic mapping and statistical aggregation to produce informative plots.

3.4.4 Matplotlib

Matplotlib is an amazing visualization library in Python for 2D plots of arrays. Matplotlib is a multi-platform data visualization library built on NumPy arrays and designed to work with the broader SciPy stack. It was introduced by John Hunter in the year 2002. One of the greatest benefits of visualization

is that it allows us visual access to huge amounts of data in easily digestible visuals. Matplotlib consists of several plots like line, bar, scatter, histogram etc.

3.4.5 PIL

The PIL or Pillow package in Python provides a way to process images in a program. We can open an image, manipulate the image for different use, and can use it to visualize the data as well. In order to use the PIL package in Tkinter, you've to install the Python Pillow library in your environment.

3.4.6 Numpy

NumPy is a Python library used for working with arrays. It also has functions for working in domain of linear algebra, fourier transform, and matrices. It is an open source project and you can use it freely.

3.4.7 Pandas

Pandas is an open-source library that is built on top of NumPy library. It is a Python package that offers various data structures and operations for manipulating numerical data and time series. It is mainly popular for importing and analyzing data much easier. Pandas is fast and it has high-performance & productivity for users.

3.4.8 Glob

The glob module is a useful part of the Python standard library. glob (short for global) is used to return all file paths that match a specific pattern. We can use glob to search for a specific file pattern, or perhaps more usefully, search for files where the filename matches a certain pattern by using wildcard characters.

3.4.9 Itertools

Python's Itertool is a module that provides various functions that work on iterators to produce complex iterators. This module works as a fast, memory-efficient tool that is used either by themselves or in combination to form iterator algebra.

CHAPTER-4

SYSTEM REQUIREMENTS SPECIFICATION

4.1 SOFTWARE REQUIREMENTS

- Operating system: Windows 10
- Programming Language: Python 3.6
- vscode
- Dependencies
 1. NumPy
 2. tensorflow
 3. Sklearn
 4. Keras
 5. OpenCV

4.2 HARDWARE REQUIREMENTS

- Processor: Intel processor
- RAM:8GB
- Disk space: 1 TB
- CPU:4GB
- Graphic card

CHAPTER-5

SYSTEM DESIGN

5.1 INTRODUCTION

Facial Expression Recognition (FER) can be seen as a second step to face detection mechanism. Humans can express emotions through facial expressions which is a part of nonverbal communication.

When a machine communicates with people, FER can give more affinities and personalized service to people depending on their emotions, which eventually increase the confidence and trust in people. Various ways in which we can express the emotions, such as facial expressions, voices, physiological signals, and text. Machines capture the expressions through camera, videos. Facial Expressions can be classified as: surprise, happy, neutral, angry, sad, disgust, and fear.

5.2 SYSTEM MODEL

The goal of our Project is to build an accurate model using “Convolution Neural Networks” Algorithm. The goal is to identify the different types of facial expressions. Subsequently training of the model is done and tested. A great way to use deep learning to classify images is to build a Convolutional Neural Network (CNN). The Keras library in Python makes it pretty simple to build a CNN. Computers see images using pixels. Pixels in images are usually related. For example, a certain group of pixels may signify an edge in an image or some other pattern. Convolutions use this to help identify images.

A Convolution multiplies a matrix of pixels with a filter matrix or „kernel“ and sums up the multiplication values. Then the convolution slides over to the next pixel and repeats the same process until all the image pixels have been covered.

Convolutional Neural Networks are very similar to ordinary Neural Networks they are made up of neurons that have learnable weights and biases. Each neuron receives some inputs, performs a dot product and optionally follows it with a non-linearity.

Regular Neural Nets don’t scale well to full images. Consider a image of size $32 \times 32 \times 3$ (32 wide, 32 high, 3 color channels), so a single fully-connected neuron in a first hidden layer of a regular Neural Network would have $32 \times 32 \times 3 = 3072$ weights. This amount still seems manageable, but clearly this fully-connected structure does not scale to larger images. For example, an image of more respectable size, e.g. $200 \times 200 \times 3$, would lead to neurons that have $200 \times 200 \times 3 = 120,000$ weights. Moreover, anyone want to have several such neurons, so the parameters would add up quickly! Clearly, this full connectivity is wasteful and the huge number of parameters would quickly lead to overfitting. Convolutional Neural Networks take advantage of the fact that the input consists of images and they constrain the architecture in a more sensible way. In particular, unlike a regular Neural Network, the layers of a ConvNet have neurons arranged in 3 dimensions: width, height, depth. For example, the input image with dimensions $X \times Y \times Z$ (width, height, depth respectively), the neurons in a layer will

only be connected to a small region of the layer before it, instead of all of the neurons in a fully-connected manner, the final output layer would have dimensions $(1,1,C)$, because by the end of the ConvNet architecture, it will reduce the full image into a single vector of class scores, arranged along the depth dimension.

Layers in Convolutional Neural Network:

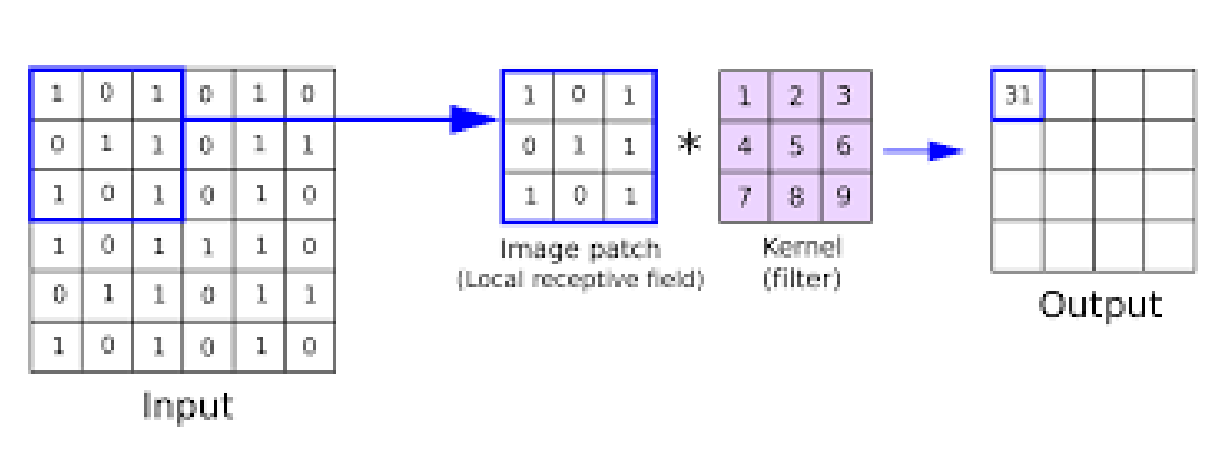
- Convolutional layer will compute the output of neurons that are connected to local regions in the input, each computing a dot product between their weights and a small region they are connected to in the input volume.
- RELU layer will apply an element wise activation function, such as the $\max(0,x)$ thresholding at zero.
- POOL layer will perform a down sampling operation along the spatial dimensions (width, height).
- FULLY connected layer will compute the class scores, resulting in volume of size $[1 \times 1 \times X]$, where each of the X numbers correspond to a class score.

Convolution Layer

When dealing with high-dimensional inputs such as images, as seen above it is impractical to connect neurons to all neurons in the previous volume. Instead, it will connect each neuron to only a local region of the input volume. The spatial extent of this connectivity is a hyperparameter called the receptive field of the neuron (equivalently this is the filter size).

The extent of the connectivity along the depth axis is always equal to the depth of the input volume. It is important to emphasize again this asymmetry in how to treat the spatial dimensions (width and height) and the depth dimension: The connections are local in space (along width and height), but always full along the entire depth of the input volume.

Example 1: For example, suppose that the input volume has size $[32 \times 32 \times 3]$, (e.g. an RGB CIFAR-10 image). If the receptive field (or the filter size) is 5×5 , then each neuron in the Conv Layer will have weights to a $[5 \times 5 \times 3]$ region in the input volume, for a total of $5 \times 5 \times 3 = 75$ weights. Notice that the extent of the connectivity along the depth axis must be 3, since this is the depth of the input volume.



5.1 CONVOLUTIONAL-LAYER REPRESENTATION

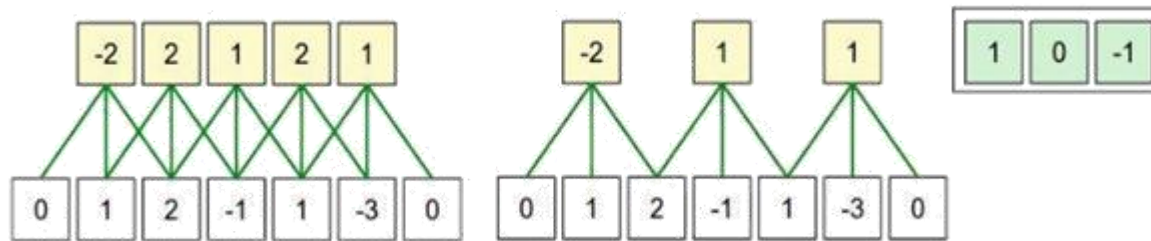
Spatial arrangement:

It is clearly explained about the connectivity of each neuron in the Conv Layer to the input volume, but haven't yet discussed how many neurons there are in the output volume or how they are arranged. Three hyperparameters control the size of the output volume: the **depth**, **stride** and **zero-padding**.

1. First, the **depth** of the output volume is a hyper parameter: it corresponds to the number of filters that are likely to be used, each learning to look for something different in the input. For example, if the first Convolutional Layer takes as input the raw image, then different neurons along the depth dimension may activate in presence of various oriented edges, or blobs of color. A set of neurons that are all looking at the same region of the input as a **depth column**.
2. Second, specification of the **stride** with to slide the filter. When the stride is 1 then , move the filters one pixel at a time. When the stride is 2 (or uncommonly 3 or more, though this is rare in practice) then the filters jump 2 pixels at a time as it slides around them. This will produce smaller output volumes spatially.
3. Sometimes it will be convenient to pad the input volume with zeros around the border. The size of this **zero-padding** is a hyper parameter. The nice feature of zero padding is that it will allow us to control the spatial size of the output volumes.

The spatial size can be computed to the size of the output volume as a function of the input volume size (W), the receptive field size of the Conv Layer neurons (F), the stride with which they are applied (S), and the amount of zero padding used (P) on the border. You can convince yourself that the

correct formula for calculating how many neurons “fit” is given by $(W - F + 2P)/S + 1$. For example, for a 7x7 input and a 3x3 filter with stride 1 and pad 0, it will result a 5x5 output. With stride 2 it will result a 3x3 output.



5.2 SPATIAL ARRANGEMENT

In this example there is only one spatial dimension (x-axis), one neuron with a receptive field size of $F = 3$, the input size is $W = 5$, and there is zero padding of $P = 1$. **Left:** The neuron strided across the input in stride of $S = 1$, giving output of size $(5 - 3 + 2)/1 + 1 = 5$. **Right:** The neuron uses stride of $S = 2$, giving output of size $(5 - 3 + 2)/2 + 1 = 3$. Notice that stride $S = 3$ could not be used since it wouldn't fit neatly across the volume. In terms of the equation, this can be determined since $(5 - 3 + 2) = 4$ is not divisible by 3. The neuron weights are in this example $[1, 0, -1]$ (shown on very right), and its bias is zero. These weights are shared across all yellow neurons. In the example above on left, note that the input dimension was 5 and the output dimension was equal: also 5. This worked out so because our receptive fields were 3 and used zero padding of 1. If there was no zero-padding used, then the output volume would have had spatial dimension of only 3, because that is how many neurons would have “fit” across the original input. In general, setting zero padding to be $P = (F - 1)/2$ when the stride is $S = 1$ ensures that the input volume and output volume will have the same size spatially.

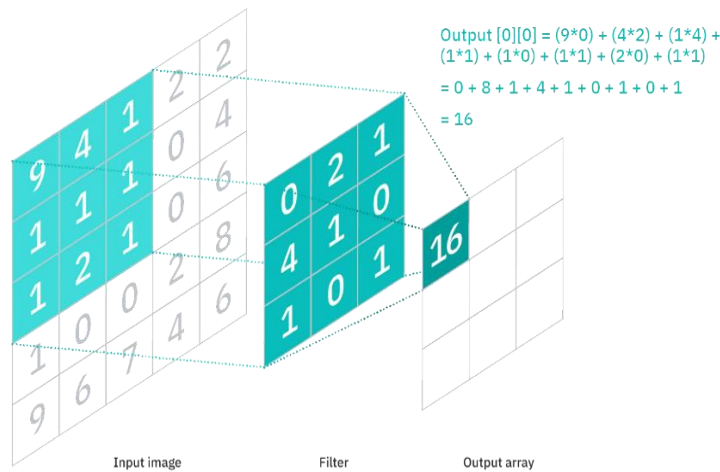
Parameter Sharing:

Parameter sharing scheme is used in Convolutional Layers to control the number of parameters. Using the real-world example above, it is clear that there are $55 \times 55 \times 96 = 290,400$ neurons in the first Conv Layer, and each has $11 \times 11 \times 3 = 363$ weights and 1 bias. Together, this adds up to $290,400 \times 364 = 105,705,600$ parameters on the first layer of the ConvNet alone. Clearly, this number is very high. It

turns out that this can dramatically reduce the number of parameters by making one reasonable assumption: That if one feature is useful to compute at some spatial position (x,y) , then it should also be useful to compute at a different position (x_2,y_2) . In other words, denoting a single 2-dimensional slice of depth as a **depth slice** (e.g. a volume of size $[55 \times 55 \times 96]$ has 96 depth slices, each of size $[55 \times 55]$), now, constrain the neurons in each depth slice to use the same weights and bias. With this parameter sharing scheme, the first Conv Layer in our example would now have only 96 unique set of weights (one for each depth slice), for a total of $96 \times 11 \times 11 \times 3 = 34,848$ unique weights, or 34,944 parameters (+96 biases). Alternatively, all 55×55 neurons in each depth slice will now be using the same parameters. In practice during backpropagation, every neuron in the volume will compute the gradient for its weights, but these gradients will be added up across each depth slice and only update a single set of weights per slice.

To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyper parameters:
 - Number of filters K ,
 - their spatial extent F ,
 - t stride S ,
 - the amount of zero padding P .
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
 - $W_2 = (W_1 - F + 2P) / S + 1$
 - $H_2 = (H_1 - F + 2P) / S + 1$ (i.e. width and height are computed equally by symmetry)
 - $D_2 = K$

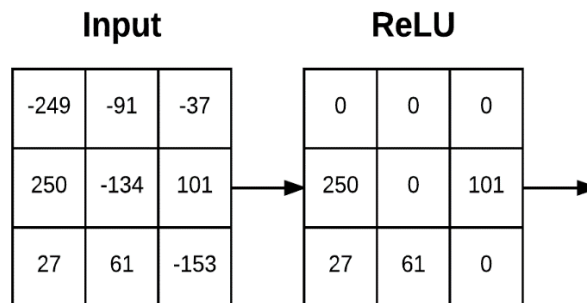


5.3 CONVOLUTION OPERATION IMPLEMENTATION

ReLU Layer

ReLU means Rectified Linear Unit, ReLU is the most used activation function in the world right now. Since, it is used in almost all the Convolutional Neural Networks or deep learning. As you can see, the ReLU is half rectified (from bottom). $f(z)$ is zero when z is less than zero and $f(z)$ is equal to z when z is above or equal to zero. Range can be maintained as $\max(0, z)$.

But the issue is that all the negative values become zero immediately which decreases the ability of the model to fit or train from the data properly. That means any negative input given to the ReLU activation function turns the value into zero immediately in the graph, which in turns affects the resulting graph by not mapping the negative values appropriately.



5.4 ReLU FUNCTIONALITY

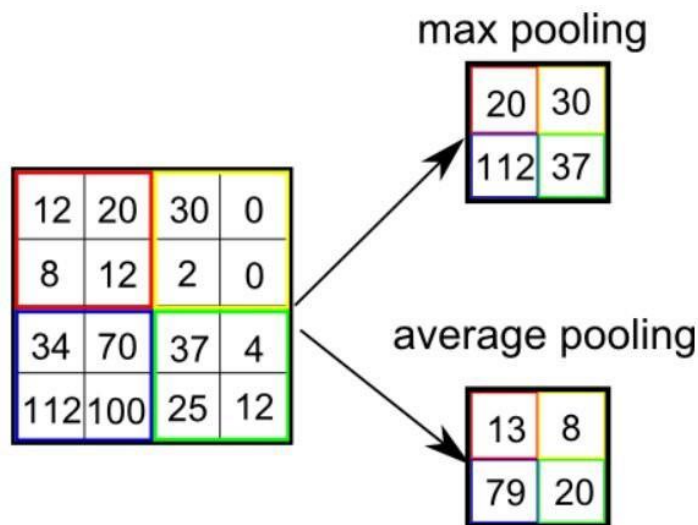
Pooling Layer

It is common to periodically insert a Pooling layer in-between successive Conv layers in a ConvNet

architecture. Its function is to progressively reduce the spatial size of the representation to reduce the amount of parameters and computation in the network, and hence to also control overfitting. The Pooling Layer operates independently on every depth slice of the input and resizes it spatially, using the MAX operation. The most common form is a pooling layer with filters of size 2x2 applied with a stride of 2 downsamples every depth slice in the input by 2 along both width and height, discarding 75% of the activations. Every MAX operation would in this case be taking a max over 4 numbers (little 2x2 region in some depth slice). The depth dimension remains unchanged. More generally, the pooling layer:

- Produces a volume of size $W2 \times H2 \times D2$ where:
 - $W2 = (W1 - F) / S + 1$
 - $H2 = (H1 - F) / S + 1$
 - $D2 = D1$
- Introduces zero parameters since it computes a fixed function of the input
- For Pooling layers, it is not common to pad the input using zero-padding.

It is worth noting that there are only two commonly seen variations of the max pooling layer found in practice: A pooling layer with $F=3, S=2$ (also called overlapping pooling), and more commonly $F=2, S=2$. Pooling sizes with larger receptive fields are too destructive.



5.5 POOLING LAYER

Fully Connected Layer

It is worth noting that the only difference between FC and CONV layers is that the neurons in the CONV layer are connected only to a local region in the input, and that many of the neurons in a CONV volume share parameters.

- For any CONV layer there is an FC layer that implements the same forward function. The weight matrix would be a large matrix that is mostly zero except for at certain blocks (due to local connectivity) where the weights in many of the blocks are equal (due to parameter sharing).
- Conversely, any FC layer can be converted to a CONV layer. For example, an FC layer with $K=4096$ that is looking at some input volume of size $7 \times 7 \times 512$ can be equivalently expressed as a CONV layer with $F=7, P=0, S=1, K=4096$. In other words, it is also like setting the filter size to be exactly the size of the input volume, and hence the output will simply be $1 \times 1 \times 4096$ since only a single depth column “fits” across the input volume, giving identical result as the initial FC layer.

FC->CONV conversion.

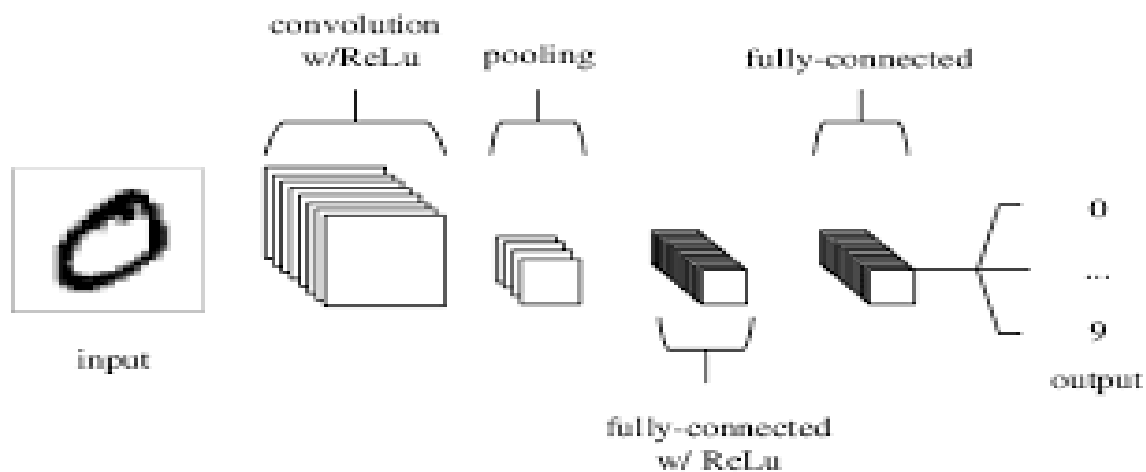
Of these two conversions, the ability to convert an FC layer to a CONV layer is particularly useful in practice. Consider a ConvNet architecture that takes a $224 \times 224 \times 3$ image, and then uses a series of CONV layers and POOL layers to reduce the image to an activations volume of size $7 \times 7 \times 512$. From there, an AlexNet uses two FC layers of size 4096 and finally the last FC layers with 1000 neurons that compute the class scores. It can be converted into each of these three FC layers to CONV layers as described above:

- Replace the first FC layer that looks at $[7 \times 7 \times 512]$ volume with a CONV layer that uses filter size $F=7$, giving output volume $[1 \times 1 \times 4096]$.
- Replace the second FC layer with a CONV layer that uses filter size $F=1$, giving output volume $[1 \times 1 \times 4096]$
- Replace the last FC layer similarly, with $F=1$, giving final output $[1 \times 1 \times 1000]$

Each of these conversions could in practice involve manipulating (e.g. reshaping) the weight matrix W in each FC layer into CONV layer filters. It turns out that this conversion allows us to “slide” the original ConvNet very efficiently across many spatial positions in a larger image, in a single forward pass. For example, if 224x224 image gives a volume of size $[7 \times 7 \times 512]$ - i.e., a reduction by 32, then forwarding an image of size 384x384 through the converted architecture would give the equivalent volume in size $[12 \times 12 \times 512]$, since $384/32 = 12$.

5.3 SYSTEM ARCHITECTURE

Putting all it together:

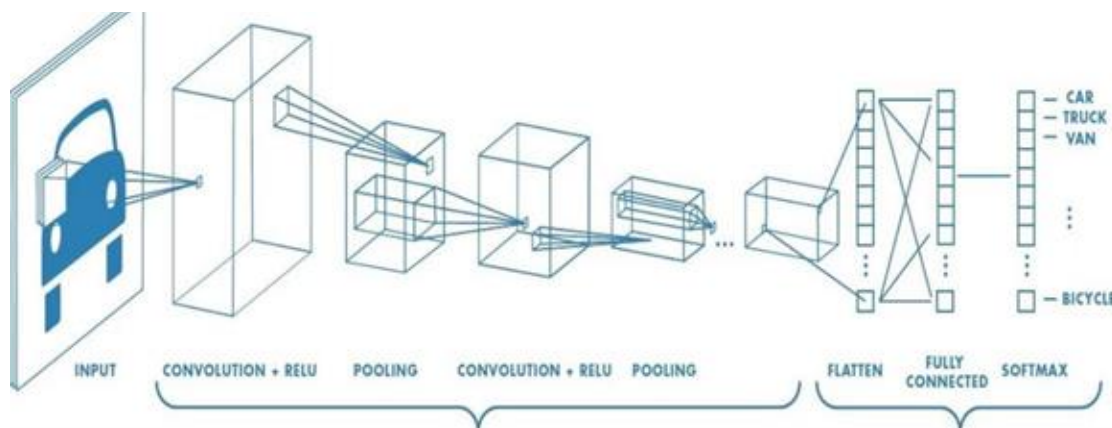


5.5 CNN ARCHITECTURE

- Pass an input image to the first Convolutional layer. The convoluted output is obtained as an activation map. The filters applied in the Convolution layer extract relevant features from the input image to pass further.
- Each filter shall give a different feature to aid the correct class prediction. In case it is needed to retain the size of the image, use same padding(zero padding), otherwise valid padding is used since it helps to reduce the number of features.
- Pooling layers are then added to further reduce the number of parameters
- Several Convolution and pooling layers are added before the prediction is made. Convolutional layer help in extracting features. Going deeper in the network more specific features are extracted as compared to a shallow network where the features extracted are more generic.

- The output layer in a CNN as mentioned previously is a fully connected layer, where the input from the other layers is flattened and sent so as to transform the output into the number of classes as desired by the network.
- The output is then generated through the output layer and is compared to the output layer for error generation. A loss function is defined in the fully connected output layer to compute the mean square loss. The gradient of error is then calculated.
- The error is then backpropagated to update the filter(weights) and bias values.
- One training cycle is completed in a single forward and backward pass.

CNN is now the go-to model on every image related problem. In terms of accuracy they blow competition out of the water. It is also successfully applied to recommender systems, natural language processing and more. The main advantage of CNN compared to its predecessors is that it automatically detects the important features without any human supervision. For example, given many pictures of cats and dogs it learns distinctive features for each class by itself.



5.6 CNN MODEL EVALUATION

Working of Convolutional Neural Network:

- Step 1: Convolution Operation.
- Step 2: Pooling.
- Step 3: Flattening.
- Step 4: Full Connected layer

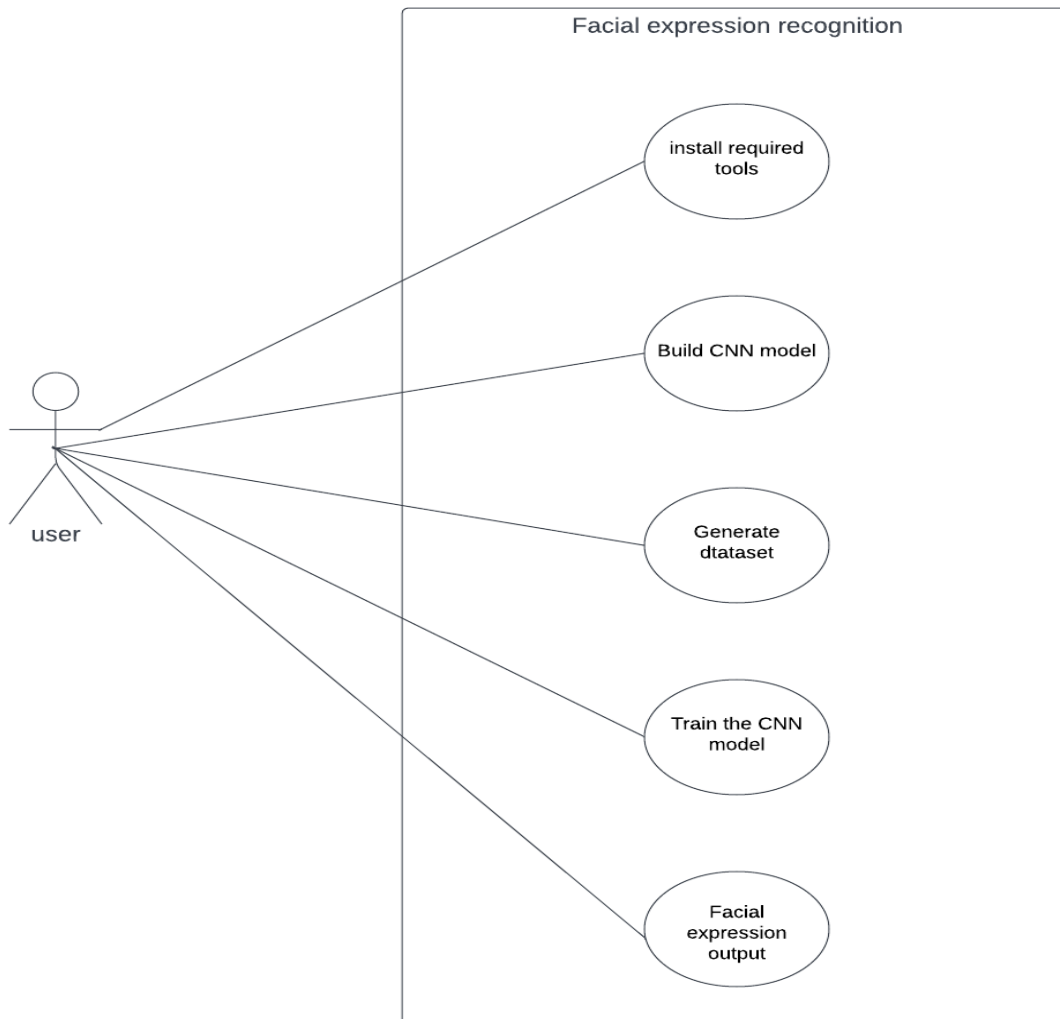
5.4 UML DIAGRAMS

A diagram is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships). You draw diagrams to visualize a system from different perspectives, so a diagram is a projection into a system. UML includes nine diagrams.

The UML diagrams are categorized into structural diagrams, behavioral diagrams, and also interaction overview diagrams. Structural diagrams depict a static view or structure of a system. It is widely used in the documentation of software architecture. It embraces class diagrams, composite structure diagrams, component diagrams, deployment diagrams, object diagrams, and package diagrams. Behavioral diagrams portray a dynamic view of a system or the behavior of a system, which describes the functioning of the system. It includes use case diagrams, state diagrams, and activity diagrams. It defines the interaction within the system. Interaction diagrams are a subclass of behavioral diagrams that give emphasis to object interactions and also depicts the flow between various use case elements of a system. In simple words, it shows how objects interact with each other and how the data flows within them. It consists of communication, interaction overview, sequence, and timing diagrams.

5.4.1 Use Case Diagram:

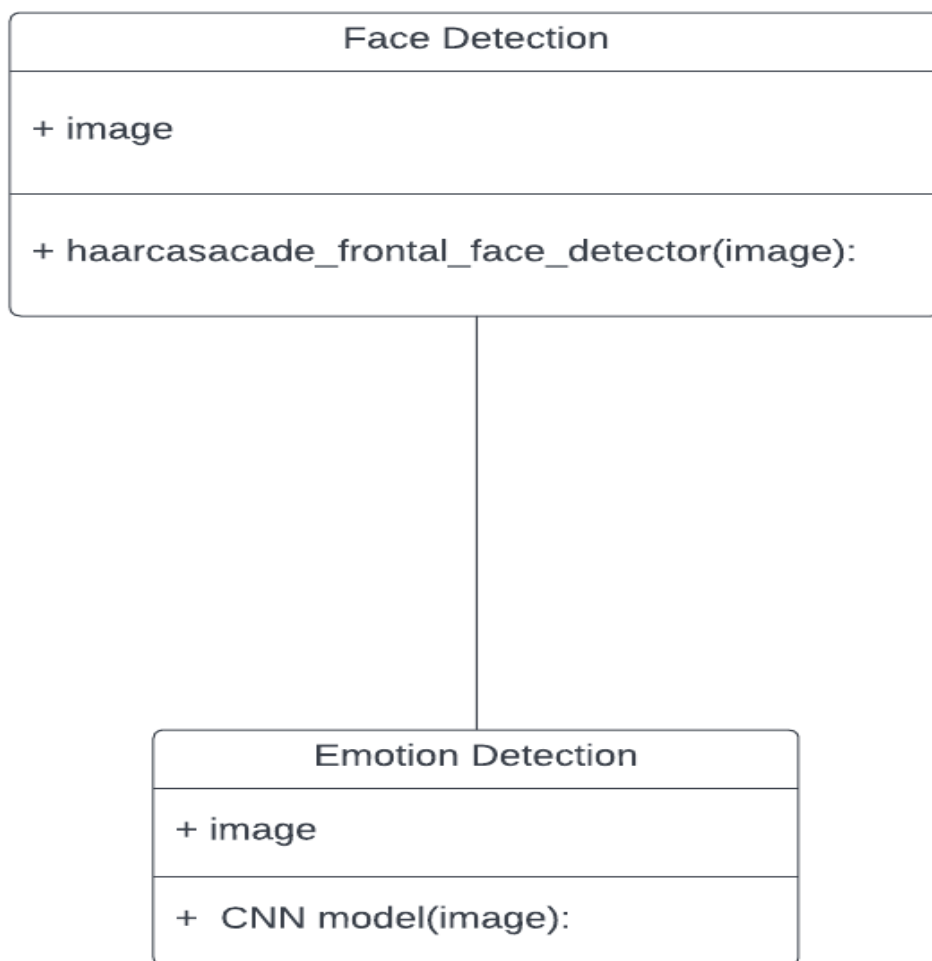
A use case diagram is a type of behavioral diagram defined in UML and creates from use case analysis. The main purpose of a use case diagram is to show what the system functions are performed for which actors. Roles of actors in the system can be depicted.



5.8 USE CASE DIAGRAM

5.4.2 Class Diagram:

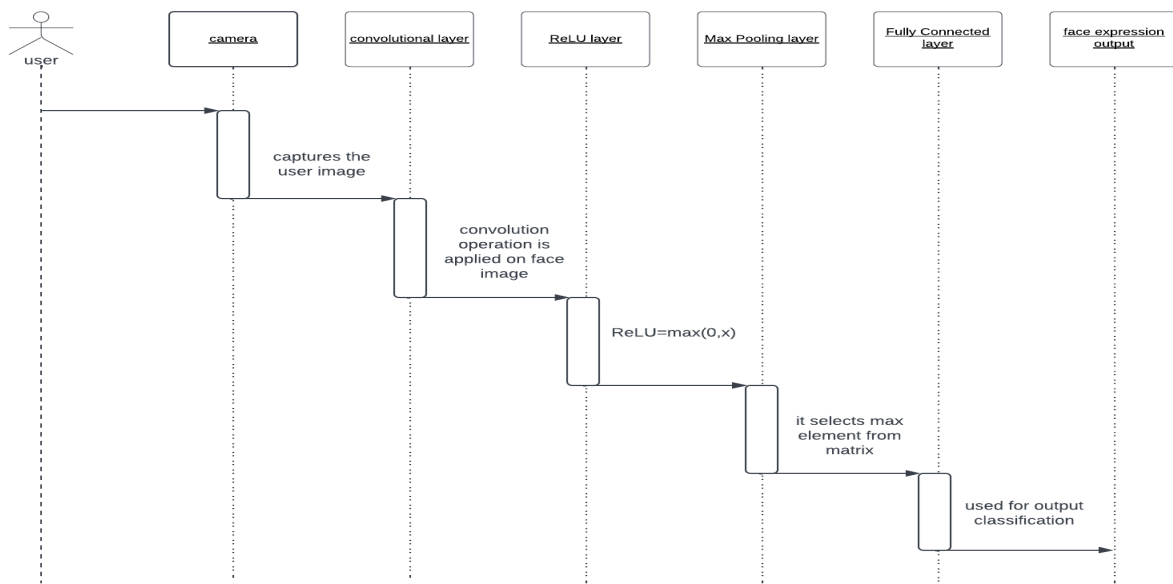
A class diagram shows a set of classes, interfaces and collaborations and their relationships. These diagrams are the most common diagrams found in modelling object-oriented systems. Class diagrams address the static design view of a system. Class diagrams that include active classes address the static process view of a system.



5.9 CLASS DIAGRAM

5.4.3 Sequence Diagram:

A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario.



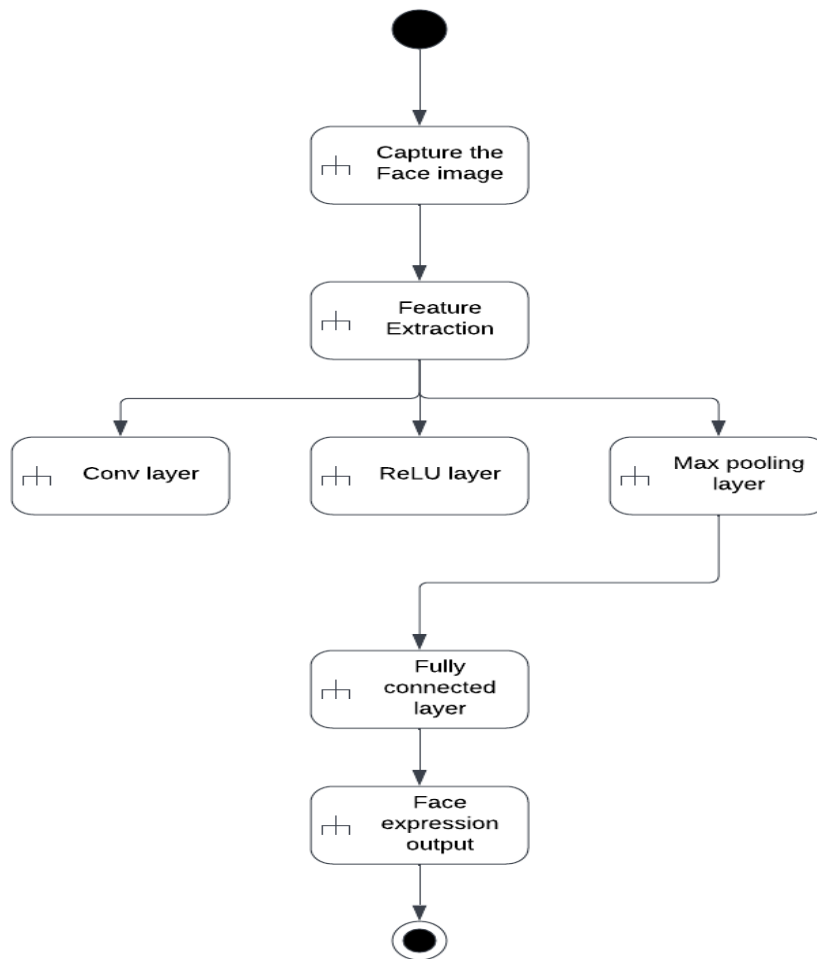
5.10 SEQUENCE DIAGRAM

5.4.4 Activity Diagram:

Activity diagram is another important diagram in UML to describe the dynamic aspects of the system.

Activity diagram is basically a flowchart to represent the flow from one activity to another activity.

The activity can be described as an operation of the system.



5.14 ACTIVITY DIAGRAM

CHAPTER-6

IMPLEMENTATION

6.1 TECHNOLOGY DESCRIPTION

The technologies used for our project are as follows:

6.1.1 OpenCV

OpenCV (Open Source Computer Vision Library) is an open source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in the commercial products. Being a BSD-licensed product, OpenCV makes it easy for businesses to utilize and modify the code.

The library has more than 2500 optimized algorithms, which includes a comprehensive set of both classic and state-of-the-art computer vision and machine learning algorithms. These algorithms can be used to detect and recognize faces, identify objects, classify human actions in videos, track camera movements, track moving objects, extract 3D models of objects, produce 3D point clouds from stereo cameras, stitch images together to produce a high resolution image of an entire scene, find similar images from an image database, remove red eyes from images taken using flash, follow eye movements, recognize scenery and establish markers to overlay it with augmented reality, etc. OpenCV has more than 47 thousand people of user community and estimated number of downloads exceeding [18 million](#). The library is used extensively in companies, research groups and by governmental bodies.

Along with well-established companies like Google, Yahoo, Microsoft, Intel, IBM, Sony, Honda, Toyota that employ the library, there are many startups such as Applied Minds, VideoSurf, and Zeitera, that make extensive use of OpenCV. OpenCV's deployed uses span the range from stitching streetview images together, detecting intrusions in surveillance video in Israel, monitoring mine equipment in China, helping robots navigate and pick up objects at Willow Garage, detection of swimming pool drowning accidents in Europe, running interactive art in Spain and New York, checking runways for debris in Turkey, inspecting labels on products in factories around the world on to rapid face detection in Japan.

It has C++, Python, Java and MATLAB interfaces and supports Windows, Linux, [Android](#) and Mac OS. OpenCV leans mostly towards real-time vision applications and takes advantage of MMX and SSE instructions when available. A full-featured [CUDA](#) and [OpenCL](#) interfaces are being actively developed right now. There are over 500 algorithms and about 10 times as many functions that compose or support those algorithms. OpenCV is written natively in C++ and has a templated interface that works seamlessly with STL containers.

6.1.2 numpy

NumPy is the fundamental package for scientific computing in Python. It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more.

At the core of the NumPy package, is the *ndarray* object. This encapsulates *n*-dimensional arrays of homogeneous data types, with many operations being performed in compiled code for performance. There are several important differences between NumPy arrays and the standard Python sequences:

- NumPy arrays have a fixed size at creation, unlike Python lists (which can grow dynamically). Changing the size of an *ndarray* will create a new array and delete the original.
- The elements in a NumPy array are all required to be of the same data type, and thus will be the same size in memory. The exception: one can have arrays of (Python, including NumPy) objects, thereby allowing for arrays of different sized elements.
- NumPy arrays facilitate advanced mathematical and other types of operations on large numbers of data. Typically, such operations are executed more efficiently and with less code than is possible using Python's built-in sequences.
- A growing plethora of scientific and mathematical Python-based packages are using NumPy arrays; though these typically support Python-sequence input, they convert such input to NumPy arrays prior to processing, and they often output NumPy arrays. In other words, in order to efficiently use much (perhaps even most) of today's scientific/mathematical Python-based software, just knowing how to use Python's built-in sequence types is insufficient - one also needs to know how to use NumPy arrays.

6.1.3 Keras

Keras is an API designed for human beings, not machines. Keras follows best practices for reducing cognitive load: it offers consistent & simple APIs, it minimizes the number of user actions required for common use cases, and it provides clear & actionable error messages. It also has extensive documentation and developer guides. Keras is the most used deep learning framework among top-5 winning teams on Kaggle. Because Keras makes it easier to run new experiments, it empowers you to try more ideas than your competition, faster. And this is how you win. It is an industry-strength framework that can scale to large clusters of GPUs or an entire TPU pod. It's not only possible; it's easy. You can export Keras models to JavaScript to run directly in the browser, to TF Lite to run on iOS, Android, and embedded devices. It's also easy to serve Keras models as via a web API. It is a central part of the tightly-connected TensorFlow 2 ecosystem, covering every step of the machine learning workflow, from data management to hyperparameter training to deployment solutions.

6.1.4 Tensorflow

TensorFlow is an open-source software library. TensorFlow was originally developed by researchers and engineers working on the Google Brain Team within Google's Machine Intelligence research organization for the purposes of conducting machine learning and deep neural networks research, but the system is general enough to be applicable in a wide variety of other domains as well! TensorFlow was developed by the Google Brain team for internal Google use in research and production. The initial version was released under the Apache License 2.0 in 2015. Google released the updated version of TensorFlow, named TensorFlow 2.0, in September 2019. TensorFlow can be used in a wide variety of programming languages, most notably Python, as well as Javascript, C++, and Java. This flexibility lends itself to a range of applications in many different sectors. TensorFlow computations are expressed as stateful dataflow graphs. The name TensorFlow derives from the operations that such neural networks perform on multidimensional data arrays, which are referred to as tensors. During the Google I/O Conference in June 2016, Jeff Dean stated that 1,500 repositories on GitHub mentioned TensorFlow, of which only 5 were from Google. As TensorFlow's market share among research

papers was declining to the advantage of PyTorch. TensorFlow 2.0 introduced many changes, the most significant being TensorFlow eager, which changed the automatic differentiation scheme from the static computational graph, to the "Define-by-Run" scheme originally made popular by Chainer and later PyTorch.

6.2 SAMPLE CODE

TrainEmotionDetector.py

```
# import required packages
import cv2
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Dropout, Flatten
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Initialize image data generator with rescaling
train_data_gen = ImageDataGenerator(rescale=1./255)
validation_data_gen = ImageDataGenerator(rescale=1./255)

# Preprocess all test images
train_generator = train_data_gen.flow_from_directory(
    'data/train',
    target_size=(48, 48),
    batch_size=64,
    color_mode="grayscale",
    class_mode='categorical')

# Preprocess all train images
validation_generator = validation_data_gen.flow_from_directory(
    'data/test',
    target_size=(48, 48),
    batch_size=64,
    color_mode="grayscale",
    class_mode='categorical')

# create model structure
emotion_model = Sequential()

emotion_model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(48, 48, 1)))
```

```

emotion_model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
emotion_model.add(Dropout(0.25))

emotion_model.add(Conv2D(128, kernel_size=(3, 3), activation='relu'))
emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
emotion_model.add(Conv2D(128, kernel_size=(3, 3), activation='relu'))
emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
emotion_model.add(Dropout(0.25))

emotion_model.add(Flatten())
emotion_model.add(Dense(1024, activation='relu'))
emotion_model.add(Dropout(0.5))
emotion_model.add(Dense(7, activation='softmax'))

cv2ocl.setUseOpenCL(False)

emotion_model.compile(loss='categorical_crossentropy', optimizer=Adam(lr=0.000
1, decay=1e-6), metrics=['accuracy'])

# Train the neural network/model
emotion_model_info = emotion_model.fit_generator(
    train_generator,
    steps_per_epoch=28709 // 64,
    epochs=50,
    validation_data=validation_generator,
    validation_steps=7178 // 64)

# save model structure in json file
model_json = emotion_model.to_json()
with open("emotion_model.json", "w") as json_file:
    json_file.write(model_json)

# save trained model weight in .h5 file
emotion_model.save_weights('emotion_model.h5')

```

TestEmotionDetector.py

```

import cv2
import numpy as np
from tensorflow.keras.models import model_from_json

```

```

emotion_dict = {0: "Angry", 1: "Disgusted", 2: "Fearful", 3: "Happy", 4: "Neutral", 5: "Sad", 6: "Surprised"}

# load json model
json_file = open("C:\\Users\\rajek\\Desktop\\second\\emotion_model.json", 'r')
loaded_model_json = json_file.read()
json_file.close()
emotion_model = model_from_json(loaded_model_json)

# load weights into new model
emotion_model.load_weights("emotion_model.h5")
print("Loaded model from disk")

img=cv2.imread("C:\\Users\\rajek\\Downloads\\img1.jpeg")

while True:
    # Find haar cascade to draw bounding box around face#
    gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY);
    face_detector = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
    faces=face_detector.detectMultiScale(gray,1.1,4)

    # take each face available on the camera and Preprocess it
    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x, y-50), (x+w, y+h+10), (0, 255, 0), 4)
        roi_gray_frame = gray[y:y + h, x:x + w]
        cropped_img = np.expand_dims(np.expand_dims(cv2.resize(roi_gray_frame,
(48, 48)), -1), 0)

        # predict the emotions
        emotion_prediction = emotion_model.predict(cropped_img)
        maxindex = int(np.argmax(emotion_prediction))
        cv2.putText(img, emotion_dict[maxindex], (x+5, y-20), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0), 2, cv2.LINE_AA)

    cv2.imshow('Emotion Detection', img)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

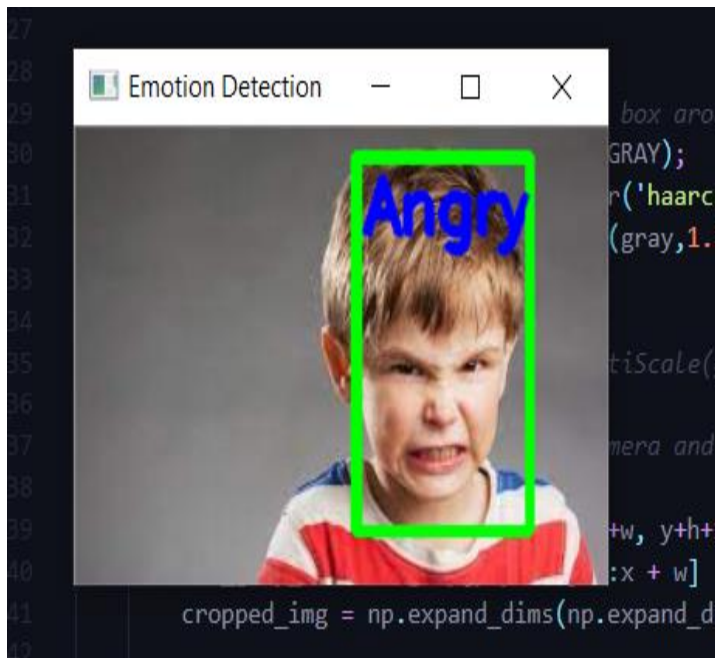
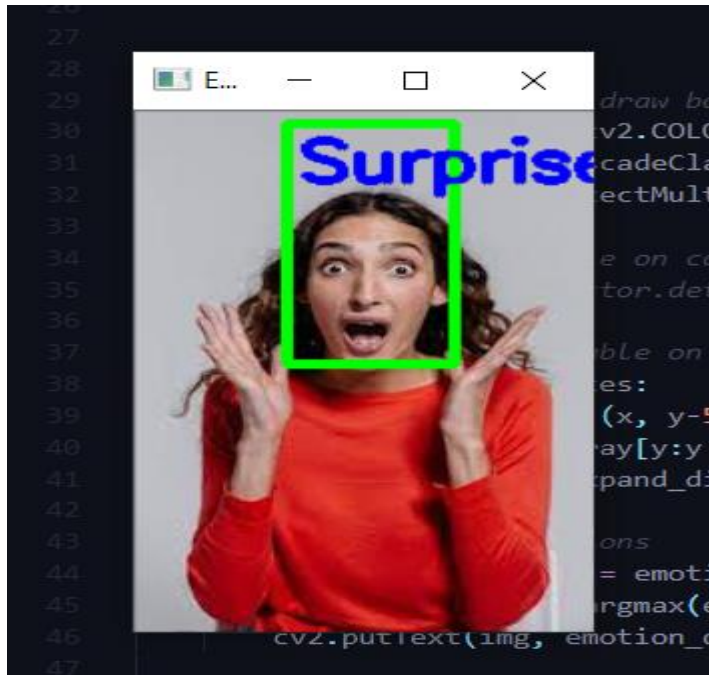
cv2.destroyAllWindows()

```

CHAPTER-7

OUTPUT SCREENS

7.1 OUTPUT SCREENSHOT



CHAPTER-8

TESTING

8.1 INTRODUCTION TO TESTING

The development of software includes series of productive activities and testing is an important activity of them. This phase is a critical element of software quality assurance and represents the ultimate review of specification, coding and testing.

Software testing is a procedure of implementing software or the application to identify the defects or bugs. For testing an application or software, we need to follow some principles to make our product defects free, and that also helps the test engineers to test the software with their effort and time.

The test engineer will test the application to make sure that the application is bug or defects free. While doing testing, we can only identify that the application or software has any errors. The primary purpose of doing testing is to identify the numbers of unknown bugs with the help of various methods and testing techniques because the entire test should be traceable to the customer requirement, which means that to find any defects that might cause the product failure to meet the client's needs.

By doing testing on any application, we can decrease the number of bugs, which does not mean that the application is defect-free because sometimes the software seems to be bug-free while performing multiple types of testing on it. But at the time of deployment in the production server, if the end-user encounters those bugs which are not found in the testing process.

Testing is a context-dependent principle states that we have multiple fields such as e-commerce websites, commercial websites, and so on are available in the market. There is a definite way to test the commercial site as well as the e-commerce websites because every application has its own needs, features, and functionality. To check this type of application, we will take the help of various kinds of testing, different technique, approaches, and multiple methods. Therefore, the testing depends on the context of the application.

The main objectives of testing are as follows:

- Testing is a process of executing a program with the intent of finding an error.
- A good test case is one that has a high probability of finding an undiscovered error.

A successful test is one uncovers an undiscovered error.

8.2 TYPES OF TESTING

Testing can be done in different ways. Some of the types of testing are mentioned below. The main purpose of any type of test is to systematically uncover different classes of errors and do so with a minimum amount of time and effort.

Types of testing:

- 8.2.1 Unit testing
- 8.2.2 Integration testing
- 8.2.3 Regression testing
- 8.2.4 System testing
- 8.2.5 Performance Testing
- 8.2.6 Alpha testing
- 8.2.7 Beta testing

Testing can be done manually or by testing tools. There are several testing tools for different software.

8.2.1 Unit Testing

It is a method by which individual units of source code, sets of one or more computer modules together with associated control data, usage procedures and operating procedures, are tested to determine if they are fit for use. Unit testing focuses verification effort on the smallest unit of software design-the software component or module. The unit test is white-box oriented. The unit testing is implemented in every module of student attendance management System. By giving correct manual input to the system, the data's are stored in database and retrieved. If a required module needed to access input or get the output, it should be checked for the accessibility and result is to be displayed else error message should be displayed.

8.2.2 Integration Testing

It is the phase in software testing in which individual software modules are combined and tested as a group. Integration testing takes as its input modules that have been unit tested, groups them in larger aggregates, applies tests defined in an integration test plan to those aggregates, and delivers as its output the integrated system ready for system testing.

8.2.3 Regression Testing

Regression testing is any type of software testing that seeks to uncover new software bugs, or regressions, in existing functional and non-functional areas of a system after changes such as enhancements, patches or configuration changes, have been made to them.

8.2.4 System Testing

System testing of software or hardware is testing conducted on a complete integrated system to evaluate the system's compliance with its specified requirements. System testing is actually a series of different tests whose primary purpose is to fully exercise the computer-based system.

8.2.5 Performance Testing

Performance testing is designed to test the run-time performance of software within the context of an integrated system. Performance testing occurs throughout all steps in the testing process. Even at the unit level, the performance of an individual module may be assessed as white-box tests are conducted. This project reduces manual work and helps in getting students attendance status in much faster. No extra time is required waiting for the results, as soon as the correct data is entered the resulted will be displayed in a few milliseconds.

8.2.6 Alpha Testing

Alpha testing is simulated or actual operational testing by potential users/customers or an independent test team at the developer's site. Alpha testing is often employed for off-the-shelf software as a form of internal acceptance testing, before the software goes to beta testing.

8.2.7 Beta Testing

Beta testing comes after alpha testing and can be considered a form of external user acceptance testing. Versions of the software, known as beta versions, are released to a limited outside of the programming team. The software is released to groups of people so that further testing can ensure the product has few faults or bugs. Sometimes beta versions are made available to the open public to increase the feedback field to a maximal number of future users.

Each module can be tested using the following two strategies:

- (1) Internal program logic is exercised using —White box || test case design techniques.
- (2) Software requirements are exercised using —Black box || test case design techniques.

In both cases, the intent is to find the maximum number of errors with the Minimum amount of effort and time.

❖ Black box Testing

In this strategy some test cases are generated as input conditions that execute all functional requirements for the program. This testing is used to find errors in the following categories:

- Incorrect or missing functions
- Interface errors
- Errors in data structure or external database access
- Performance errors
- Initialization and termination errors

In this testing only the output is checked for correctness. The logical flow of the data is not checked.

❖ White box Testing

In this testing, test cases are generated on the logic of each module by drawing flow graphs of that module and logical decisions are tested on all the cases. It is a method of testing software tests internal structures or workings of an application as opposed to its functionality(i.e. Black box testing). In white box testing an internal perspective of the system, as well as programming skills, are used to design test cases. The tester chooses inputs to exercise paths throw the code and determine appropriate outputs. This is analogous to testing nodes in a circuit.

White box Testing can be applied at the unit, integration, system levels of the software testing process. Although traditional testers tended to think of White box Testing as being one at the unit level.

8.3 SAMPLES TEST CASES

8.3.1 Test code:

```
img=cv2.imread("C:\\Users\\rajek\\Downloads\\img1.jpeg")

# take each face available on the camera and Preprocess it
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y-50), (x+w, y+h+10), (0, 255, 0), 4)
    roi_gray_frame = gray[y:y + h, x:x + w]
```

```

        cropped_img = np.expand_dims(np.expand_dims(cv2.resize(roi_gray_frame,
(48, 48)), -1), 0)

        # predict the emotions
        emotion_prediction = emotion_model.predict(cropped_img)
        maxindex = int(np.argmax(emotion_prediction))
        cv2.putText(img, emotion_dict[maxindex], (x+5, y-
20), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0), 2, cv2.LINE_AA)

cv2.imshow('Emotion Detection', img)

```

8.3.2 Output:

Test case is an object for execution for other modules in the architecture does not represent any interaction by itself. A test case is a set of sequential steps to execute a test operating on a set of predefined inputs to produce certain expected outputs.

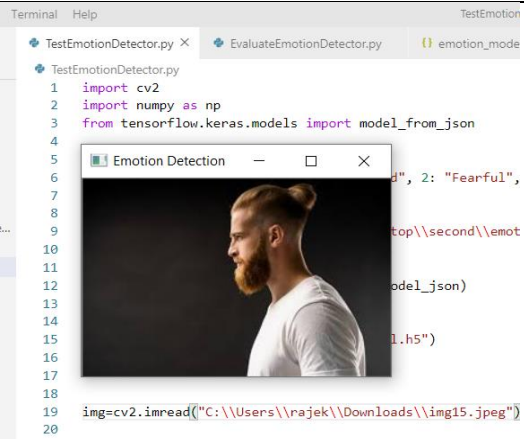
There are two types of test cases: -manual and automated.

- A **manual test case** is executed manually while an automated test case is executed using automation. In system testing, test data should cover the possible values of each parameter based on the requirements. Since testing every value is impractical, a few values should be chosen from each equivalence class. An equivalence class is a set of values that should all be treated the same. Ideally, test cases that check error conditions are written separately from the functional test cases and should have steps to verify the error messages and logs. Realistically, if functional test cases are not yet written, it is ok for testers to check for error conditions when performing normal functional test cases. It should be clear which test data, if any is expected to trigger errors.
- An **automated test case** is a software testing technique that performs using special automated testing software tools to execute a test case suite. Manual Testing is performed by a human sitting in front of a computer carefully executing the test steps. The automation testing software can also enter test data into the System Under Test, compare expected and actual results and generate detailed test reports. Software Test Automation demands considerable investments of money and resources. Successive development cycles will require execution of same test suite repeatedly. It is possible to record this test suite and re-play it as required. Once the test suite is automated, no human intervention is required. The goal of Automation is to reduce the number of test cases to be run manually and not to eliminate Manual Testing altogether.

Test Case ID, Test Scenario, Test Case Description, Test Steps, Prerequisite, Test Data, Expected Result, Test Parameters, Actual Result, Environment Information, Comments are some of the basic test case parameters to be followed.

While developing the test cases for the above scenario, the test cases are documented the following way. In the below example, the first case is a pass scenario while the second case is a FAIL. If the expected result doesn't match with the actual result, then we log a defect. The defect goes through the defect life cycle and the testers address the same after fix.

Test Cases		Predicted Output	Actual Output
		Surpsised	Surprised
		Happy	Happy
		Fear	Fear

Test Cases	Predicted Output	Actual Output
	Face should not be detected	Face is not detected
	Angry	Angry

8.1 SAMPLE TEST CASES TABLE

CHAPTER-9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 CONCLUSION AND FUTURE ENHANCEMENTS

we have built a facial expression recognition method using a CNN model which extracts facial features effectively. Compared to traditional methods, the proposed method can automatically learn pattern features and reduce the incompleteness caused by artificial design features. The proposed method directly inputs the image pixel value through training sample image data. Autonomous learning can implicitly acquire more abstract feature expression of the image. The training process of the proposed method uses appropriate initialization of weights which has a great impact on the updating of weights. Our extensive experimental analysis shows that compared to the past literatures, the proposed algorithm can improve the recognition rate of facial expressions in complex background to a certain extent. Compared to FRR-CNN and R-CNN models, the convergence speed of proposed model is much faster in complex background environments. Also, the proposed method achieves a higher recognition rate. Facial expressions captured in reality may have various noises, such as face posture, occlusion, and blurring. To address this concern, as a future work, we will investigate more robust models which satisfy real conditions. We will also focus on how to reduce the complexity of network structure, and will try to recognize dynamic expressions with 3D convolution technology.

REFERENCES

REFERENCES

- [1]. P. Ekman, & W. V. Friesen. Constants across cultures in the face and emotion. *Journal of Personality and Social Psychology*, 17(2), 124-129. (1971)
- [2] Challenges in representation learning: Facial expression recognition challenge. (2013)
<http://www.kaggle.com/c/challenges-in-representationlearning-facial-expression-recognition-challenge>.
- [3] Zhiding Yu & Cha Zhang. (2015). Image based Static Facial Expression Recognition with Multiple Deep Network Learning. *Proceedings of the 2015 ACM on International Conference on Multimodal Interaction* . 435- 442.
- [4] Raghuvarshi, A., & Choksi, V. (2016). Facial Expression Recognition with Convolutional Neural Networks. *CS231n Course Projects*.
- [5] Abhinav Dhall, Roland Goecke, Simon Lucey, and Tom Gedeon. (2011). Static Facial Expressions in Tough Conditions: Data, Evaluation Protocol And Benchmark, First IEEE International Workshop on Benchmarking Facial Image Analysis Technologies BeFIT, IEEE International Conference on Computer Vision ICCV2011, Barcelona, Spain, 6-13 November 2011.
- [6] Ebrahimi Kahou, S., Michalski, V., Konda, K., Memisevic, R., and Pal, C. (2015). Recurrent neural networks for emotion recognition in video. *Proceedings of the 2015 ACM on International Conference on Multimodal Interaction*. 467-474. ACM.
- [7] Abhinav Dhall, Roland Goecke, Simon Lucey, Tom Gedeon. (2012). Collecting Large, Richly Annotated Facial-Expression Databases from Movies, *IEEE Multimedia*, 19(3):3441, July 2012
- [8] Q. V. Le, N. Jaitly, and G. E. Hinton. (2015). A simple way to initialize recurrent networks of rectified linear units. *arXiv preprint arXiv:1504.00941*.
- [9] A. Mollahosseini, D. Chan and M. H. Mahoor. (2016). Going deeper in facial expression recognition using deep neural networks. *2016 IEEE Winter Conference on Applications of Computer Vision (WACV)*, Lake Placid, NY, 1-10.
- [10] Li M., Xu H., Huang X., Song Z., Liu X. and Li X. (2018). Facial Expression Recognition with Identity and Emotion Joint Learning. *IEEE Transactions on Affective Computing*. 1-1.
- [11] Tan L., Zhang K., Wang K., Zeng X., Peng X. and Qiao Y. (2017) Group emotion recognition with individual facial emotion CNNs and global image based CNNs. *Proceedings of the 19th ACM International Conference on Multimodal Interaction - ICMI 2017*, 549-552. ACM.
- [12] A. Krizhevsky, I. Sutskever, and G. E. Hinton. (2012). Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 10971105.

- [13] Lucey, P., Cohn, J. F., Kanade, T., Saragih, J., Ambadar, Z., & Matthews, I. (2010). The Extended Cohn-Kanade Dataset (CK+): A complete expression dataset for action unit and emotion-specified expression. Proceedings of the Third International Workshop on CVPR for Human Communicative Behavior Analysis (CVPR4HB 2010), San Francisco, USA, 94-101.
- [14] D. V. Sang, N. Van Dat and D. P. Thuan, "Facial expression recognition using deep convolutional neural networks," 2017 9th International Conference on Knowledge and Systems Engineering (KSE), Hue, 2017, pp. 130-135.
- [15] Open Source Computer Vision. Face Detection using HaarCascades:
https://docs.opencv.org/3.4.1/d7/d8b/tutorial_py_face_detection.html
- [16] Chollet, Francois and others (2015). Keras: <https://keras.io/> [17] Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python, JMLR

PAPER **PUBLICATION**

[INFOCOMP Journal of Computer Science] Submission Acknowledgement



External

Inbox



André Pimenta Freire 9:02 pm

to me



Be careful with this message.

Gmail could not verify that it actually came from infocomp.dcc.ufla@gmail.com. Avoid clicking links, downloading attachments or replying with personal information.

Report spam

Report phishing

RAJESH KANCHARAPOO:

Thank you for submitting the manuscript, "Facial Expression Recognition using Convolutional neural network" to INFOCOMP Journal of Computer Science. With the online journal management system that we are using, you will be able to track its progress through the editorial process by logging in to the journal web site:

Manuscript URL: <https://infocomp.dcc.ufla.br/index.php/Infocomp/authorDashboard/submission/2256>

Username: raj14299

If you have any questions, please contact me. Thank you for considering this journal as a venue for your work.

18:03

11.0 KB/S VoLTE 4G 37



ugccare.unipune.ac.in/A

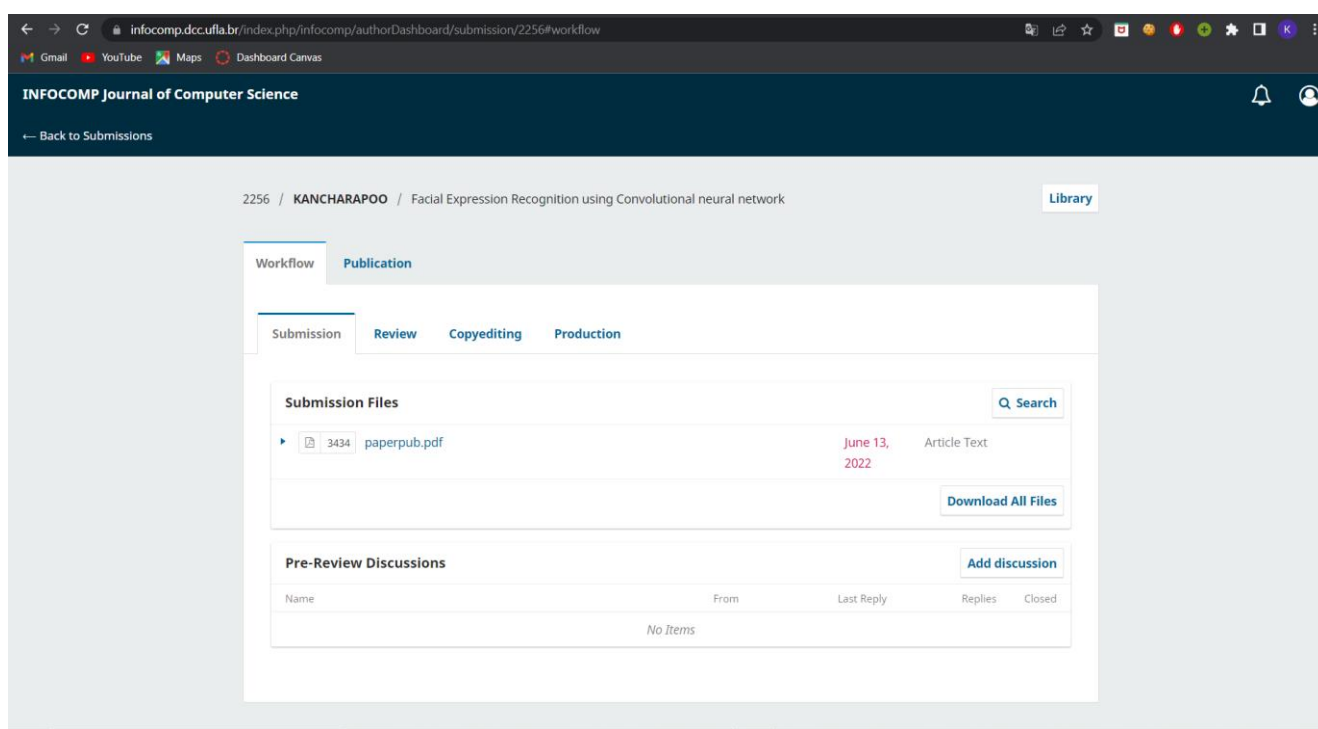
2



≡ UGC-CARE List

Journal Details

Journal Title (in English Language)	INFOCOMP Journal of Computer Science
Publication Language	English
Publisher	Department of Computer Science, Federal University of Lavras
ISSN	NA
E-ISSN	1982-3363
Discipline	Science
Subject	Computer Science (all)
Focus Subject	General Computer Science



Facial Expression Recognition using convolutional neural network

K.Rajesh ¹, M.V.Dinesh ², M.Abdul Rahman ³

¹ Department of Computer Science and Engineering at Raghu Engineering College, Vishakhapatnam, India.

² Department of Computer Science and Engineering at Raghu Engineering College, Vishakhapatnam, India.

³ Department of Computer Science and Engineering at Raghu Engineering College, Vishakhapatnam, India.

E-mail address: 18981a0570@raghuengcollege.in, 18981a0591@raghuengcollege.in, 18981a0596@raghuengcollege.in

Abstract: Emotions play an important role in conveying the message to others. Human beings can easily recognize emotions. Computers are able to interact with the human beings more naturally because of the technologies like AI and machine learning if we could build the system which can identify the human emotions then it would help us in counselling and other health related fields. This paper elaborates development of convolutional neural network which is a deep learning based technique to recognize the facial expression. The aim is to create convolutional neural network model which classifies the facial image into one of the seven facial expressions. The dataset that we have used for training the CNN model is 48x48 grey scale images from kaggle 2013 facial expression recognition. The facial expression recognition dataset is divided into two folders test and train where test and train are further divided into separate folder each containing one of the seven types of the facial expression recognition dataset.

Keywords: Facial Expression Recognition (FER), Convolutional neural network (CNN), Deep learning.

1. INTRODUCTION

Human beings communicate with each other in the form of speech, gestures and emotions. As such systems that can recognize the same are in great demand in many fields. With respect to artificial intelligence, a computer will be able to interact with humans much more naturally if they are capable of understanding human emotion. It would also help during counseling and other health care related fields. In an E-Learning system, the presentation style may be varied depending on the student's state. However in many cases, static emotion detection is not very useful. It is essential to know the user's feelings over a period of time in a live environment. Thus, the paper proposes a model that is aimed at real-time facial emotion recognition. For real-time purposes, facial emotion recognition has a number of applications. Facial emotion recognition could be used in conjunction with other systems to provide a form of safety. For instance, ATMs could be set up such that they won't dispense money when the user is scared. In the gaming industry, emotion-aware games can be developed which could vary the difficulty of a level depending on the player's emotions. It also has uses in video game testing. At present, players usually give some form of verbal or written feedback. Using facial emotion recognition, the number of testers can be increased to accommodate people who use different languages or people who are unable to cohesively state their opinions on the game. By judging their expressions during different points of the game, a

general understanding of the game's strong and weak points can be discerned. Emotions can also be gauged while a viewer watches ads to see how they react to them. This is especially helpful since ads do not usually have feedback mechanisms apart from tracking whether the ad was watched and whether there was any user interaction. Software for cameras can use emotion recognition to take photos whenever a user smiles.

2. LITERATURE REVIEW

Facial Expression Recognition:

In recent years, researchers have made considerable progress in developing automatic expression classifiers (Shima Alizadeh and Azar Fazel, no date). In 19th century, Charles Darwin has contributed important aspect in facial expression analysis which is directly linked with current science of automatic facial expression recognition. Charles Darwin's worked on the general principals of expressions in humans and animals (SRIMANI P.K. and HEGDE R., 2012). In his work he has grouped several kinds of expressions in groups like: shame, shyness, modesty, anxiety, grief, low spirits, dejection, despair joy, tender feelings, high spirits, love, devotion, reflection, meditation, ill-temper, anger disdain, contempt, sulkiness, determination, hatred, disgust, astonishment, fear, guilt, pride, surprise, horror, self-attention. The observation of the study says the enlargement of the muscles around eyes and mouth varies

as per the changes in the emotions e.g.: depression, surprise, happiness, etc. (Darwin C., no date).

Various range of CNN, modelled and trained for facial emotion recognition are evaluated in (A. M. Badshaah, J. Ahmed and S. W. Baek, 2017). Facial emotion Recognition is drawing its own importance in the research field. Facial emotion recognition is inspected and analyzed on all research areas (A. Routrey, M. Swaen and P. Kabisetpathy, 2018). Emotion is recognized from facial images using filter banks and Deep CNN (K.-Y. Hueng et al., 2016), this leads to high accuracy rate which implies that deep learning can also be used for facial expression detection.

Two important objectives of Facial Expression Recognition and Analysis (FERA) are feature extraction and expression classification. Ming et. al. defined three phases of FER: facial image registration, feature extraction, and FER (Nithya Roopa. S, 2019). The facial expression recognition has two methods: classification and regression methods. Deep learning is showing promising results, including in FER (Nithya Roopa. S, 2019). This paper contributes a Deep Learning and CNN approaches with multiclass classification problem so it is categorical cross entropy loss function to track highest accuracy.

Convolutional Neural Networks (CNN) are neural network architecture which has multilayers (D Y Liliana, 2018). CNN input and output are array vectors known as feature map. The type of input decides the array dimension value. As an example, audio input has one dimensional array as well as text input; image has 2D array. The output feature map describes the feature extracted from the input. CNN has three key layers: convolutional filter layer, pooling/subsampling layer, and classification layer (Nithya Roopa. S, 2019).

3. EXISTING WORK

Haar functions is used for face, eyes and mouth detection, edge detection method for extracting the eyes correctly and bezier curves is applied to approximate the extracted regions. Then a set of distances for varied face type is extended and it is serve as training input for a multilayer neural network. The novel factor of this approach consists in applying bezier curves to efficiently extract the distances between facial parts. The preclassification is done using K-means algorithm. A two layered feed forward neural network created is then used as a classifying tool for the input images. The consistency of the results is demonstrated by the median value. The method is not able to treat situations when the eyes are closed. Strong illumination variations affect the results.

4. DATASET AND FEATURES

The data consists of 48x48 pixel grayscale images of faces. The faces have been automatically registered so that the face is more or less centred and occupies about the same amount of space in each image.

The task is to categorize each face based on the emotion shown in the facial expression into one of seven categories (0=Angry, 1=Disgust, 2=Fear, 3=Happy, 4=Sad, 5=Surprise, 6=Neutral). The training set consists of 28,709 examples and the public test set consists of 3,589 examples.

Data Explorer

Version 1 (56.51 MiB)

- ▼ test
 - ▶ angry
 - ▶ disgust
 - ▶ fear
 - ▶ happy
 - ▶ neutral
 - ▶ sad
 - ▶ surprise
- ▼ train
 - ▶ angry
 - ▶ disgust
 - ▶ fear
 - ▶ happy
 - ▶ neutral
 - ▶ sad
 - ▶ surprise

Fig [1]: Datasets used are considered from Kaggle.

5. PROPOSED WORK

A Convolutional neural network is a neural network comprised of convolution layers which does computational heavy lifting by performing convolution. Convolution is a mathematical operation on two functions to produce a third function. It is to be noted that the image is not represented as pixels, but as numbers representing the pixel value. In terms of what the computer sees, there will simply just be a matrix of numbers. The convolution operation takes place on these numbers. We utilize both fully-connected layers as well as convolutional layers. In a fully-connected layer, every node is connected to every other neuron. They are the layers used in standard feedforward neural networks. Unlike the fullyconnected layers, convolutional layers are not connected to every neuron. Connections are made across localized regions. A sliding "window" is moved across the image. The size of this window is known as the kernel or the filter. They help recognise patterns in the data. For each filter, there are two main properties to consider -

padding and stride. Stride represents the step of the convolution operation, that is, the number of pixels the window moves across. Padding is the addition of null pixels to increase the size of an image. Null pixels here refers to pixels with value of 0. If we have a 5x5 image and a window with a 3x3 filter, a stride of 1 and no padding, the output of the convolutional layer will be a 3x3 image. This condensation of a feature map is known as pooling. In this case, "max pooling" is utilized. Here, the maximum value is taken from each sliding window and is placed in the output matrix. Convolution is very effective in image recognition and classification compared to a feed-forward neural network. This is because convolution allows to reduce the number of parameters in a network and take advantage of spatial locality. Further, convolutional neural networks introduce the concept of pooling to reduce the number of parameters by downsampling. Applications of Convolutional neural networks include image recognition, self-driving cars and robotics. CNN is popularly used with videos, 2D images, spectrograms, Synthetic Aperture Radars..

6. METHODOLOGY AND IMPLEMENTATION

The convolutional neural network consists of six two-dimensional convolutional layers, two max pooling layers and two fully connected layers. Max pooling uses the maximum value from each of a cluster of neurons at the prior layer. This reduces the dimensionality of the output array. The input to the network is a preprocessed face of 48 x 48 pixels. The model was developed based on the observation of the performance of the previous models. It was decided to go with a deeper network over a wide one. The advantage of using more layers is that it prevents memorization. A wide but shallow network memorizes well but does not generalize well. Multi-layer networks learn features at levels of abstractions allowing them to generalize well. The number of layers were selected so as to maintain a high level of accuracy while still being fast enough for real-time purposes. The proposed CNN differs from a simple CNN in that it uses 4 more convolutional layers and each of its convolutional layers differ in filter size. In addition, it utilized max pooling and dropout more effectively in order to minimize overfitting. The network consists of two convolutional layers with a filter size of 64 each. This is then followed by a max pooling layer. A dropout of rate 0.25 is applied to reduce overfitting. This is followed by a sequence of four convolutional layers. The first two have a filter size of 128 each and the latter two have a filter size of 256 each. A single max pooling layer follows these four layers with a dropout of rate 0.25. In order to convert the output into a single dimensional vector, the output of the previous layers was flattened. A fully connected layer with a L2 regularizer of penalty of 0.001 is then used along with an additional dropout of rate 0.5. Finally, a fully connected layer with a softmax

activation function serves as the output layer. The kernel size, that is, the width and height of the 2D convolutional window is set to 3 x 3 for all convolutional layers. Each max pooling layer is two dimensional and uses a pool size of 2 x 2. This halves the size of the output after each pooling layer. All the layers bar the output layer used a ReLU activation function. The ReLU activation function is used here due to benefits such as sparsity and a reduced likelihood of vanishing gradient. The softmax activation function was used in the final output layer to receive the predicted probability of each emotion.

Algorithms Used

Convolutional neural network:

Convolutional neural networks are mainly used for image recognition and object detection

CNN detects patterns and make sense of the input image.

The Following are the layers in the CNN

1. Convolutional layer
2. ReLU layer
3. Pooling layer
4. Fully connected layer

1.Convolutional Layer:

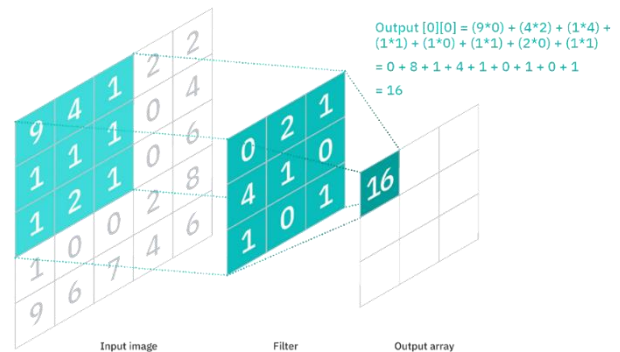
Convolutional layer is the first layer in the CNN architecture. This layer is applied to extract the features from the given input image. A convolution operation is done in this layer which generates the feature map.

This layer requires three components:

Input data

Filter

Feature map



2.ReLU Layer:

ReLU stands for rectified linear unit. This layer ensures that every negative value in the feature map is replaced with zero.

ReLU function : $f(x)=\max(0,x)$

- It does operations element wise on the feature map.
- It gives the output of the rectified feature map.

Input			ReLU		
-249	-91	-37	0	0	0
250	-134	101	250	0	101
27	61	-153	27	61	0

3.Pooling Layer:

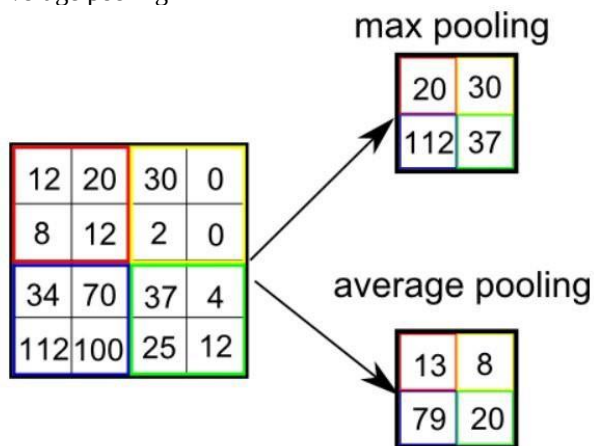
Pooling layer usually comes after convolutional layer. Pooling layer is used to decrease the size of the feature map to reduce the computational costs.

There are different types of Pooling operations.

Max pooling

Min pooling

Average pooling



4.Fully connected layer:

Fully connected layers are dense network of neurons and connection between every two neurons. The fully connected layers are used to classify the image into particular category. In our case fully connected layer classifies the image into one of the seven expressions. After the feature extraction has been done by the convolutional and pooling layers the feature maps are flattened and passed as input to the fully connected layer. The final fully connected layer contains the same number of neurons as the number of categories. In our case the final fully connected layer contains seven neurons as there are seven different facial expressions to classify.

7.

RESULTS

CNN model is set to 50 epochs and when trained gives accuracy 85%. The designed CNN model was able to recognize the emotions happy,sad,neutral and surprise accurately subject to 48x48 images.

User Interface Results

These user interface results are considered to show the final output for the following paper work. This helps the user to interact easily without any background knowledge how the code works. The user interface results are displayed in the below figures Fig [9] and [10].

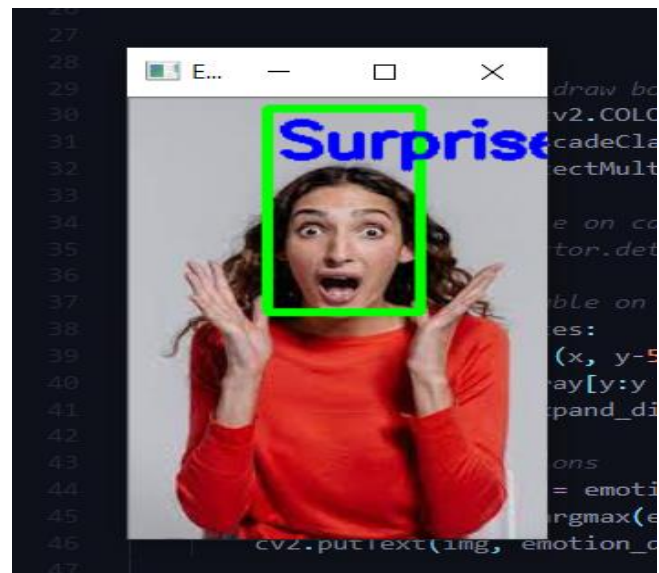


Fig [9]: User Interface to add the article.

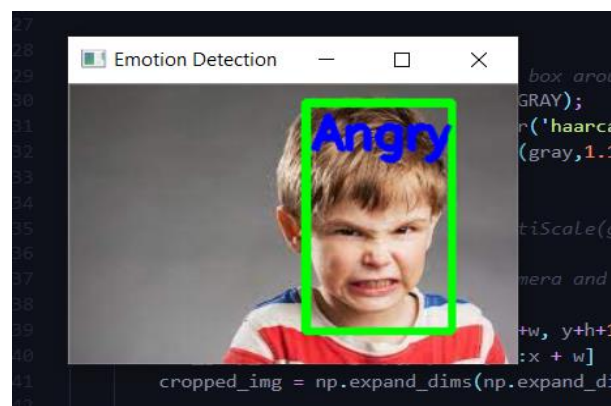


Fig [10]: Final Result that is displayed to the User

8. CONCLUSION

Using 48x48 grey-scale images from Kaggle's ICMP 2013-Facial Expression Recognition (FER) dataset, test accuracy is 85% with the above designed CNN model. The same CNN model will be used for recognizing Facial Expressions. Trained the same model on few converted grey scale 48x48 images and the results are satisfactory. The ability of the model to make predictions in effectively real-time, indicates that real world uses of facial emotion recognition is barred only by the relative inaccuracies of the model itself. In the future, an indepth analysis of the top2 predicted emotions may lead to a much more accurate and reliable system. Further training samples for the more difficult to predict emotion of disgust will definitely be required in order to perfect such a system.

REFERENCES

- [1]. P. Ekman, & W. V. Friesen. Constants across cultures in the face and emotion. *Journal of Personality and Social Psychology*, 17(2), 124-129. (1971)
- [2] Challenges in representation learning: Facial expression recognition challenge. (2013) <http://www.kaggle.com/c/challenges-in-representationlearning-facial-expression-recognition-challenge>.
- [3] Zhiding Yu & Cha Zhang. (2015). Image based Static Facial Expression Recognition with Multiple Deep Network Learning. *Proceedings of the 2015 ACM on International Conference on Multimodal Interaction* . 435-442.
- [4] Raghuvanshi, A., & Choksi, V. (2016). Facial Expression Recognition with Convolutional Neural Networks. CS231n Course Projects.
- [5] Abhinav Dhall, Roland Goecke, Simon Lucey, and Tom Gedeon. (2011). Static Facial Expressions in Tough Conditions: Data, Evaluation Protocol And Benchmark, First IEEE International Workshop on Benchmarking Facial Image Analysis Technologies BeFIT, IEEE International Conference on Computer Vision ICCV2011, Barcelona, Spain, 6-13 November 2011.
- [6] Ebrahimi Kahou, S., Michalski, V., Konda, K., Memisevic, R., and Pal, C. (2015). Recurrent neural networks for emotion recognition in video. *Proceedings of the 2015 ACM on International Conference on Multimodal Interaction*. 467-474. ACM..
- [7] Abhinav Dhall, Roland Goecke, Simon Lucey, Tom Gedeon. (2012). Collecting Large, Richly Annotated Facial-Expression Databases from Movies, *IEEE Multimedia*, 19(3):3441, July 2012
- [8] Q. V. Le, N. Jaitly, and G. E. Hinton. (2015). A simple way to initialize recurrent networks of rectified linear units. *arXiv preprint arXiv:1504.00941*.
- [9] A. Mollahosseini, D. Chan and M. H. Mahoor. (2016). Going deeper in facial expression recognition using deep neural networks. 2016 IEEE Winter Conference on Applications of Computer Vision (WACV), Lake Placid, NY, 1-10.
- [10] Li M., Xu H., Huang X., Song Z., Liu X. and Li X. (2018). Facial Expression Recognition with Identity and Emotion Joint Learning. *IEEE Transactions on Affective Computing*. 1-1.
- [11] Tan L., Zhang K., Wang K., Zeng X., Peng X. and Qiao Y. (2017) Group emotion recognition with individual facial emotion CNNs and global image based CNNs. *Proceedings of the 19th ACM International Conference on Multimodal Interaction - ICMI 2017*, 549-552. ACM.
- [12] A. Krizhevsky, I. Sutskever, and G. E. Hinton. (2012). Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 10971105.
- [13] Lucey, P., Cohn, J. F., Kanade, T., Saragih, J., Ambadar, Z., & Matthews, I. (2010). The Extended Cohn-Kanade Dataset (CK+): A complete expression dataset for action unit and emotion-specified expression. *Proceedings of the Third International Workshop on CVPR for Human Communicative Behavior Analysis (CVPR4HB 2010)*, San Francisco, USA, 94-101.
- [14] D. V. Sang, N. Van Dat and D. P. Thuan, "Facial expression recognition using deep convolutional neural networks," 2017 9th International Conference on Knowledge and Systems Engineering (KSE), Hue, 2017, pp. 130-135.
- [15] Open Source Computer Vision. Face Detection using HaarCascades: https://docs.opencv.org/3.4.1/d7/d8b/tutorial_py_face_detection.html
- [16] Chollet, François and others (2015). Keras: <https://keras.io/> [17] Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python, JMLR